

Appunti di Algoritmi per l'ingegneria

Giacomo Simonetto

Secondo semestre 2025-26

Sommario

Appunti del corso di Algoritmi per l'Ingegneria della facoltà di Ingegneria Informatica dell'Università di Padova.

Indice

1	Cenni di storia dell'algoritmica	3
2	Formalizzazioni dei concetti base	4
2.1	Algoritmo	4
2.2	Problema computazionale	4
2.3	Dimostrazione per induzione	6
2.4	Formule matematiche utili	7
3	Paradigma Divide and Conquer	8
3.1	Proprietà di sottostruttura	8
3.2	Analisi della correttezza	9
3.3	Albero della ricorsione	9
3.4	Analisi della complessità e formulone	10
3.5	Master theorem	11
3.6	Applicazioni del divide and conquer	13
4	Esponenziazione con esponente intero	14
4.1	Algoritmo ovvio basato sulla definizione	14
4.2	Algoritmo efficiente basato sul divide and conquer	14
5	Moltiplicazione di polinomi - Karatsuba	15
5.1	Prodotto tra polinomi usando la definizione	15
5.2	Prodotto tra polinomi con approccio divide and conquer	15
5.3	Prodotto di polinomi con l'algoritmo di Karatsuba (1962)	16
6	Moltiplicazione di matrici - Strassen	17
6.1	Prodotto di matrici usando la definizione	17
6.2	Prodotto di matrici con approccio divide and conquer	17
6.3	Prodotto di matrici con l'algoritmo di Strassen (1969)	18
6.4	Algoritmo ibrido per il prodotto di matrici	19
7	Moltiplicazione di matrici supercircolanti - Tr. di Hadamard	21
7.1	Moltiplicazioni tra matrici supercircolanti	21
7.2	Trasformata di Hadamard e Fast Hadamard Transform (FHT)	22
8	Moltiplicazione di polinomi e FFT - Fast Fourier Transform	23
8.1	Rappresentazioni di un polinomio	23
8.2	Valutazione di un polinomio	24
8.3	Interpolazione di un polinomio	25
8.4	Matrice di Fourier e radici n-esime dell'unità	26
8.5	Discrete Fourier Transform e algoritmo FFT	27
8.6	Moltiplicazione di polinomi con la DFT e padding dei coefficienti	29
8.7	Osservazioni utili per l'esame	30
9	Programmazione dinamica	31
9.1	Memoizzazione	31
9.2	Calcolo dell' n -esimo numero di Fibonacci	31
9.3	Longest Common Subsequence - LCS	34
9.4	Matrix Chain Multiplication - MCM	38
9.5	Longest Palindrome Subsequence - LPS	40
10	Paradigma Greedy	42
10.1	Proprietà della scelta greedy	42
10.2	Scheduling di attività	42
10.3	Codice di Huffman	44
10.4	(2,1)-boxing	46

1 Cenni di storia dell'algoritmica

...

2 Formalizzazioni dei concetti base

2.1 Algoritmo

Definizione formale

Un algoritmo è una macchina a stati finiti che riceve una sequenza di input, effettua una sequenza finita di transizioni tra gli stati interni della macchina e produce una sequenza di output.

Si osserva che tale definizione coincide con la definizione di una macchina di Turing.

Stopping problem

Si definisce lo “stopping problem” come la proprietà di un algoritmo di non poter continuare all’infinito. Ad esempio non esiste un algoritmo che calcoli tutte le cifre di π o che verifichi se un’equazione diofantea (o diofantina) ha soluzioni intere.

Progettazione di un algoritmo

La progettazione di un algoritmo è il processo di ideazione e formalizzazione di un algoritmo per risolvere uno specifico problema computazionale. Questo processo coinvolge:

- la conoscenza del dominio del problema ed eventuali proprietà che lo caratterizzano, in quanto potrebbero semplificare il processo di risoluzione
- la conoscenza di metodi generali di progettazione di algoritmi, come i paradigmi di programmazione

Analisi di un algoritmo

L’analisi di un algoritmo è il processo di valutazione dell’algoritmo secondo i diversi aspetti:

- correttezza: valuta se l’algoritmo porta alla soluzione corretta del problema per ogni possibile input
- efficienza: valuta il consumo di risorse in termini di passi effettuati (complessità temporale), di memoria occupata (complessità spaziale) ed eventuali altri parametri (es. consumo energetico)

2.2 Problema computazionale

Definizione formale

Un problema computazionale è composto dai seguenti tre elementi:

- un insieme $\mathcal{I}$ di istanze, che rappresentano i possibili input del problema, si nota che alcuni problemi potrebbero avere un numero infinito di istanze, per cui $\mathcal{I}$ potrebbe essere un insieme infinito
- un insieme $\mathcal{S}$ di soluzioni, che rappresentano tutti gli output del problema per ogni istanza di $\mathcal{I}$
- una relazione $\pi \subseteq \mathcal{I} \times \mathcal{S}$ che associa ogni istanza $i \in \mathcal{I}$ ad una o più soluzioni $s_j \in \mathcal{S}$ valide per quella istanza (è un sottoinsieme del prodotto cartesiano tra $\mathcal{I}$ e $\mathcal{S}$)

Rappresentazione delle istanze e delle soluzioni come stringhe

I dati di un’istanza e di una soluzione sono rappresentati come stringhe, ovvero sequenze finite di simboli. Si assumono le seguenti convenzioni:

$\Sigma = \{0, 1, \dots\}$	l’alfabeto dei simboli delle stringhe
$\Sigma^0 = \{\varepsilon\}$	l’insieme della sola stringa vuota
Σ^i	l’insieme di tutte le stringhe di lunghezza i (potenza cartesiana)
$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$	l’insieme di tutte le stringhe esclusa la stringa vuota
$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$	l’insieme di tutte le stringhe inclusa la stringa vuota
$\mathcal{I} \subseteq \Sigma^*, \quad \mathcal{S} \subseteq \Sigma^*$	gli insiemi di istanze e soluzioni sono sottoinsiemi di Σ^*

La taglia di una istanza è la quantità di informazione necessaria per rappresentarla ed è pari al numero di simboli che compongono la stringa che la rappresenta, o meglio la sua lunghezza. Per un numero intero n rappresentato in binario, la taglia è il numero di cifre che lo compongono $\approx \log_2 n$.

Osservazioni sulla rappresentazione dei dati nell'insieme delle istanze

L'insieme delle istanze $\mathcal{I}$ impone il tipo di rappresentazione in cui i dati in input devono essere codificati. Fornire i dati in una diversa rappresentazione potrebbe portare a soluzioni errate, ad esempio se l'algoritmo svolge la somma di numeri in base 10 e gli vengono forniti in base 2.

Inoltre la scelta della rappresentazione influisce sull'efficienza dell'algoritmo in quanto alcune operazioni risultano più efficienti in una rappresentazione rispetto ad un'altra. Ad esempio la convoluzione nel dominio del tempo è $O(n^2)$, mentre nel dominio della frequenza è $O(n \log n)$.

All'interno dell'algoritmo è possibile aggiungere una fase di conversione dei dati da una rappresentazione ad un'altra (ad esempio per ridurre la complessità computazionale), ma è sempre necessario specificare la rappresentazione dei dati in ingresso.

Funzioni e algoritmi

Il concetto di relazione è più generale di quello di funzione, in quanto una relazione può associare più elementi del codominio (es. soluzioni) ad un unico elemento del dominio (es. un'istanza). Una funzione, invece, può associare al massimo un elemento del codominio ad ogni elemento del dominio.

Un algoritmo che risolve un problema computazionale definisce una funzione tra l'insieme delle istanze e quello delle soluzioni. A partire da una istanza, restituirà soltanto una delle possibili soluzioni per tale istanza. È possibile, inoltre, che due algoritmi diversi risolvono lo stesso problema computazionale, ma producano soluzioni diverse a partire dalla stessa istanza.

Analisi della correttezza di un algoritmo

Un algoritmo $\mathcal{A}$ risolve un problema computazionale π se e solo se, detta A la funzione di trasferimento tra ingresso e uscita definita da $\mathcal{A}$, valgono le seguenti condizioni:

$$\begin{array}{lll} 1. \mathcal{I}_{\text{istanze}} \subseteq I_{\text{input}} & 2. \mathcal{S}_{\text{soluzioni}} \subseteq O_{\text{output}} & 3. \forall i \in \mathcal{I} \text{ vale } (i, A(i)) \in \pi \\ \text{con} & \pi \subseteq \mathcal{I}_{\text{istanze}} \times \mathcal{S}_{\text{soluzioni}} & \text{e} \quad A : I_{\text{input}} \rightarrow O_{\text{output}} \end{array}$$

L'analisi della correttezza consiste in particolare nel verificare la terza condizione.

Analisi della complessità di un algoritmo

L'analisi della complessità computazionale di un algoritmo consiste nel valutare il tempo necessario all'algoritmo per raggiungere la soluzione a partire da una data istanza generica.

La misurazione del tempo effettivo impiegato dipende molto dall'architettura della macchina su cui avviene la misurazione. Il valore ottenuto sarebbe troppo variabile e non permette di stimare in maniera corretta e generale la complessità di un algoritmo. Per questo motivo si simula l'esecuzione su random access machines con instruction set semplici e tempo di esecuzione costante per ogni istruzione, usato come unità di misura del tempo di esecuzione. Questa semplificazione permette sia di stimare la complessità di un algoritmo in maniera più generale, sia di confrontare tra loro algoritmi diversi prima di implementarli su una macchina reale.

Si effettua l'analisi al caso peggiore (worst-case) definendo come tempo impiegato da un algoritmo $\mathcal{A}$ per risolvere un'istanza i di taglia n come:

$$T_{\mathcal{A}}(n) := \sup_{x \in \Sigma^n} T_{\mathcal{A}}(x) \quad \text{con } \{x : |x| = n \wedge x \in \Sigma^n\}$$

L'analisi al caso migliore è poco utile, mentre l'analisi al caso medio è notevolmente più complessa da effettuare, in quanto richiede di conoscere a priori la distribuzione di probabilità delle varie istanze per una specifica taglia n . In alcuni casi quest'ultima viene preferita all'analisi al caso peggiore, se il caso peggiore è molto raro e mediamente (per la maggior parte dei casi) la complessità risulta inferiore, ad esempio per il quicksort o per l'algoritmo del semplice.

2.3 Dimostrazione per induzione

Assiomi di Peano dei numeri naturali

1. esiste un numero naturale 0
2. ogni numero naturale n ha un successore $s(n)$
3. numeri diversi hanno successori diversi: $n \neq m \implies s(n) \neq s(m)$
4. 0 non è il successore di nessun numero naturale: $s(n) \neq 0 \quad \forall n$
5. forma debole del principio di induzione:

$$\left[\underbrace{P(0)}_{\text{caso base}} \wedge \underbrace{\forall n (P(n) \implies P(n+1))}_{\text{passo induttivo}} \right] \implies P(n) \quad \forall n$$

Il quinto assioma può essere sostituito con il principio del minimo intero che sostiene che ogni insieme ordinabile non vuoto ha un elemento minimo. Il principio di induzione e quello del minimo intero sono equivalenti, siccome il primo implica il secondo e viceversa.

Forma forte del principio di induzione

Esiste una formulazione più forte del principio di induzione, equivalente alla forma debole enunciata sopra, ma che facilita le dimostrazioni basate su questo principio.

$$\left[\underbrace{P(0)}_{\text{caso base}} \wedge \underbrace{\forall n ((P(0) \wedge P(1) \wedge \dots \wedge P(n)) \implies P(n+1))}_{\text{passo induttivo}} \right] \implies P(n) \quad \forall n$$

Tale forma forte è ad esempio usata per dimostrare la correttezza del quicksort e negli ordinali transfiniti.

Rafforzamento dell'ipotesi induttiva

Alcune ipotesi induttive più “permissive” potrebbero non essere dimostrabili tramite il principio di induzione, anche se sono vere. Si riescono invece a dimostrare ipotesi più forti e ambiziose che implicano quelle più deboli. Ad esempio potrebbe non essere dimostrabile che $T(n) \leq n$, ma si riesce a dimostrare che $T(n) \leq n - 1$ che implica $T(n) \leq n$.

Questo fenomeno paradossale viene chiamato rafforzamento dell'ipotesi induttiva. Si spiega con il fatto che l'ipotesi induttiva compare sia nelle premesse che nelle conseguenze e, un suo rafforzamento, potrebbe sia rafforzare le conseguenze che rafforzare le premesse a sua volta. Alcune volte il rafforzamento delle premesse è maggiore rispetto a quello delle conseguenze e ciò permette di dimostrare queste ultime più facilmente.

Mappatura sui numeri naturali

Il principio di induzione è definito solo sui numeri naturali e può essere applicato solo su proprietà che ricevono come argomenti numeri naturali. Per dimostrare la correttezza di un algoritmo su un certo insieme di input (che non necessariamente coincide con l'insieme dei numeri naturali) è necessario effettuare una opportuna mappatura tra gli elementi di tale insieme e i numeri naturali. Spesso si clusterizza l'insieme degli input in base alla loro taglia e si mappa ogni cluster ad un numero naturale che corrisponde alla taglia degli input in quel cluster.

Ad esempio, si vorrebbe dimostrare una certa proprietà che vale per gli alberi binari completi, ma gli alberi binari completi non sono mappabili 1:1 con i numeri naturali. Si clusterizzano, quindi, in base alla loro taglia (ad esempio in base al numero di nodi interni) e si mappa ogni cluster ad un numero naturale equivalente proprio al numero di nodi interni. Si applica quindi il principio di induzione dimostrando che la proprietà vale per ogni cluster.

Limiti dell'induzione

Il principio di induzione si utilizza principalmente per dimostrare ipotesi formulate a priori. Non aiuta a formulare nuove ipotesi, che vanno pensate con l'aiuto di altre proprietà o intuizioni.

Induzione parametrica

L'induzione parametrica o induzione empirica è una tecnica che permette di individuare i parametri di una funzione generale o di un modello generale che si ritiene soddisfare al meglio i dati. Può essere usata, ad esempio, nell'addestramento di reti neurali. In pratica consiste nell'applicare il principio di induzione imponendo che la formula valga per il caso base e per il passo induttivo.

Ad esempio di vuole calcolare la formula analitica della seguente sommatoria:

$$s(n) = \sum_{k=1}^n k^2$$

Si notano le seguenti proprietà:

- $s(n) \leq n^3$ se tutti i valori della sommatoria fossero n^2
- $s(n) \geq n^3/8$ siccome almeno $n/2$ valori della sommatoria sono maggiori o uguali a $(n/2)^2$
- $s(n) \approx n^3/3 - 1/3$ se si tratta la sommatoria come un'integrale finito

per cui si ipotizza che la formula analitica sia un'equazione di terzo grado

$$s(n) = an^3 + bn^2 + cn + d$$

- caso base

si impone che valga il caso base $s(0) = 0$ da cui si ottiene $d = 0$

- passo induttivo

si impone che valga il passo induttivo $s(n+1) = s(n) + (n+1)^2$:

$$\begin{aligned} a(n+1)^3 + b(n+1)^2 + c(n+1) + d &= an^3 + bn^2 + cn + d + (n+1)^2 \\ an^3 + (3a+b)n^2 + (3a+2b+c)n + (a+b+c+d) &= an^3 + bn^2 + cn + d + n^2 + 2n + 1 \\ 3an^2 + (3a+2b)n + (a+b+c) &= n^2 + 2n + 1 \end{aligned}$$

da cui si ottengono le tre equazioni che determinano gli altri tre parametri a , b e c :

$$3a = 1 \Rightarrow a = 1/3 \quad 3a + 2b = 2 \Rightarrow b = 1/2 \quad a + b + c = 1 \Rightarrow c = 1/6$$

Si osserva che è possibile trovare gli stessi quattro parametri imponendo che la formula valga per quattro valori particolari di n (ad esempio $n = 0, 1, 2, 3$) e risolvendo il sistema di quattro equazioni in quattro incognite. Tale procedimento, però, senza aver osservato all'inizio che la formula è un'equazione di terzo grado, non permette di affermare che i parametri trovati valgano per ogni n . L'equazione, ad esempio, potrebbe essere di grado superiore per cui i quattro punti scelti non sono sufficienti.

2.4 Formule matematiche utili

$$a^{\log_b c} = c^{\log_b a}$$

esponenziale di un logaritmo

$$\sum_{k=0}^{n-1} r^k = \frac{r^n - 1}{r - 1}$$

serie geometrica di ragione r

$$z = a + ib \Leftrightarrow z = \rho(\cos \varphi + i \sin \varphi) \Leftrightarrow z = \rho \cdot e^{i\varphi}$$

$$\begin{cases} \rho = \sqrt{a^2 + b^2} \\ \varphi = \arctan(b/a) \end{cases} \longleftrightarrow \begin{cases} a = \rho \cos(\varphi) \\ b = \rho \sin(\varphi) \end{cases}$$

rappresentazioni di un numero complesso con passaggio tra le varie rappresentazioni

$$\sum_{k=0}^{n-1} \binom{n}{k} a^k b^{n-k} = (a+b)^n \quad \sum_{k=0}^n \binom{n}{k} = 2^n$$

binomio di Newton con caso particolare per $a = b = 1$ (sottosequenze di una stringa)

3 Paradigma Divide and Conquer

3.1 Proprietà di sottostruttura

Idea generale

Il paradigma divide and conquer (divide et impera) è un approccio che consiste nel suddividere un'istanza di grandi dimensioni in una o più istanze di dimensioni inferiori, più semplici da risolvere, e poi combinare le soluzioni di queste istanze per ottenere la soluzione dell'istanza originale.

Proprietà di sottostruttura

Dato un problema $\pi \subseteq \mathcal{I} \times \mathcal{S}$, il paradigma si compone di tre componenti:

- $A_D : \mathcal{I} \rightarrow \mathcal{I}^*$, funzione di divide che suddivide un'istanza $i \in \mathcal{I}$ di taglia $|i| > n_0$ in una sequenza di una o più istanze più piccole $\langle i_1, i_2, \dots, i_k \rangle$ di taglia $|i_j| < |i|$ per ogni j
- $A_C : \mathcal{S}^* \rightarrow \mathcal{S}$, funzione di combine che combina la sequenza di soluzioni $\langle s_1, s_2, \dots, s_k \rangle$ associate alle istanze $\langle i_1, i_2, \dots, i_k \rangle$ nella soluzione s associata all'istanza i
- $D\&C : \mathcal{I} \rightarrow \mathcal{S}$, chiamata ricorsiva che trasforma individualmente ogni istanza i_j nella corrispondente soluzione s_j , applicando eventualmente le due funzioni A_D e A_C in modo ricorsivo

Le due funzioni A_D e A_C devono soddisfare la seguente proprietà di sottostruttura:

$$\forall i \in \mathcal{I}, |i| > n_0 \text{ vale } [D(i) = \langle i_1, \dots, i_k \rangle \wedge \forall j (i_j, s_j) \in \pi \wedge C(\langle s_1, \dots, s_k \rangle) = s \implies (i, s) \in \pi]$$

Chiamata ricorsiva (conquer) e definizione del caso base

La chiamata ricorsiva $D\&C$ è definita come segue:

- se $|i| > n_0$ allora $D\&C(i) = A_C(A_D(i))$, ovvero riapplica il modello di divide and conquer
- se $|i| \leq n_0$ allora utilizza una soluzione diretta ed immediata per risolvere i

In generale il parametro n_0 è scelto in base al problema. Può essere il minimo valore per cui è possibile applicare il modello di divide and conquer, oppure un valore sufficientemente grande sotto di cui si conosce un algoritmo più efficiente per risolvere il problema.

0: **Algoritmo:** $D\&C(i)$

Input: istanza $i \in \mathcal{I}$

Output: soluzione $s \in \mathcal{S}$

```
1:  if  $|i| \leq n_0$  then                                ▷ caso base
2:       $s \leftarrow A_{\text{diretto}}(i)$                         ▷ soluzione diretta ed immediata
3:      return  $s$ 
4:  else                                                ▷ passo ricorsivo
5:       $\langle i_1, i_2, \dots, i_k \rangle \leftarrow A_D(i)$         ▷ divide
6:      for  $j \leftarrow 1$  to  $k$  do                        ▷ conquer
7:           $s_j \leftarrow D\&C(i_j)$ 
8:       $s \leftarrow A_C(\langle s_1, s_2, \dots, s_k \rangle)$     ▷ combine
9:      return  $s$ 
```

3.2 Analisi della correttezza

Proprietà di sottostruttura in funzione di un parametro intero

Per dimostrare la correttezza di un algoritmo divide and conquer, si utilizza la proprietà di sottostruttura e il principio di induzione in forma forte. Per prima cosa si esprime la proprietà di sottostruttura in funzione di un parametro intero, in modo da renderla più adatta alla dimostrazione induttiva, ovvero:

$$\forall i \in \mathcal{I} \underbrace{[(i, D\&C(i)) \in \pi]}_{P(i)} = \forall n \in \mathbb{N} \underbrace{[\forall i \in \mathcal{I} \text{ con } |i| = n \text{ vale } (i, D\&C(i)) \in \pi]}_{P(n)}$$

Dimostrazione con il principio di induzione in forma forte

Si dimostra, infine, che $P(n)$ è vera per ogni $n \in \mathbb{N}$ tramite il principio di induzione in forma forte:

- si dimostra che $P(n)$ è vera per ogni $n \leq n_0$ (caso base)
- si dimostra che se $P(n)$ è vera per tutti i valori $n \leq m$ allora anche $P(m+1)$ è vera (passo induttivo)

3.3 Albero della ricorsione

Albero della ricorsione

L'albero della ricorsione è una rappresentazione grafica che rappresenta le chiamate ricorsive di un algoritmo divide and conquer con le seguenti caratteristiche:

- ogni nodo rappresenta una chiamata ricorsiva $D\&C(i)$ per un'istanza $i \in \mathcal{I}$
- ogni nodo ha come figli i nodi che rappresentano le chiamate ricorsive $D\&C(i_j)$ per ogni istanza i_j ottenuta dalla procedura di divide $A_D(i)$
- la taglia dell'istanza dei nodi figli è inferiore alla taglia dell'istanza del nodo padre, in quanto la taglia dell'istanza diminuisce ad ogni chiamata ricorsiva
- ogni foglia rappresenta una chiamata ricorsiva $D\&C(i)$ per un'istanza $i \in \mathcal{I}$ di taglia $|i| \leq n_0$, ovvero un caso base

Esplorazione dell'albero della ricorsione

L'esplorazione dell'albero della ricorsione avviene secondo la visita in-order, ovvero prima si visita il figlio sinistro, poi il nodo padre, poi il figlio destro. In questo modo, dato il nodo dell'albero che si sta attualmente visitando, si è certi che tutti i nodi a sinistra di esso sono stati visitati, e i nodi a destra di esso sono ancora da visitare.

La costruzione della soluzione avviene secondo la visita post-order, ovvero prima si risolvono le istanze dei figli, poi viene costruita la soluzione per il nodo padre, secondo l'algoritmo DFS (Depth First Search).

Lo spazio occupato è proporzionale allo spazio richiesto da ciascuna chiamata ricorsiva e dal numero di chiamate ricorsive attive contemporaneamente, ovvero la profondità dell'albero della ricorsione.

Cenni sulla ricorsione nei primi linguaggi di programmazione

La memoria a disposizione sui primi computer era molto limitata e non era sufficiente per implementare uno stack che isolasse le variabili locali delle varie chiamate ricorsive attive. I primi linguaggi di programmazione, infatti, salvavano le variabili locali in posizioni fisse di memoria, comuni ad ogni chiamata ricorsiva. Per questo motivo, all'inizio la ricorsione non era supportata ed il suo utilizzo era fortemente scoraggiato. Il primo linguaggio di programmazione a supportare nativamente la ricorsione è stato il linguaggio LISP, sviluppato da John McCarthy nel 1958.

Ad oggi, la ricorsione è supportata nativamente da tutti i principali linguaggi di programmazione ed esistono ottimizzazioni a livello hardware (istruzioni macchina per la gestione dello stack) per renderla efficiente e non sprecare memoria.

3.4 Analisi della complessità e formulone

Equazione alle ricorrenze

L'equazione alle ricorrenze è un'equazione che esprime la complessità temporale di un algoritmo divide and conquer in funzione della complessità temporale delle chiamate ricorsive e delle operazioni di divide e combine. Di seguito è riportata la forma generale con le proprietà indicate sotto:

$$T(n) = \begin{cases} T_0(n) & \text{se } n \leq n_0 \quad - \quad \text{caso base} \\ s(n) \cdot T(f(n)) + \omega(n) & \text{se } n > n_0 \quad - \quad \text{passo ricorsivo} \end{cases}$$

- $T_0(n)$ è il costo dell'algoritmo diretto per risolvere un'istanza di taglia $n \leq n_0$
- $s(n)$ è il numero di chiamate ricorsive effettuate per risolvere un'istanza di taglia n e coincide con il numero di nodi figli di ogni nodo dell'albero della ricorsione
- $f(n)$ è la taglia della sottoistanza, generata dalla procedura di divide di un'istanza di taglia n , che viene passata alla chiamata ricorsiva
- $T(f(n))$ è il costo di ogni chiamata ricorsiva per risolvere un'istanza di taglia $f(n)$
- $\omega(n) = T_D(n) + T_C(n)$ è il costo complessivo delle procedure di divide e di combine

Per ottenere il valore analitico ed eliminare le ricorrenze è necessario risolvere l'equazione attraverso un'analisi analitica, o ricorrendo al formulone o al master theorem.

Iterate di f

Si definisce l'iterata di una funzione f come la potenza della composizione di f con se stessa.

$$\begin{cases} f^{(0)}(n) = n \\ f^{(l)}(n) = f(f^{(l-1)})(n) \quad \text{per } l > 0 \end{cases}$$

Si osserva che $f^{(i)}(n)$ corrisponde alla taglia dell'istanza all' i -esima chiamata ricorsiva. In particolare, inizialmente $f^{(i)}(n) = n$ per $i = 0$ e progressivamente diminuisce ad ogni iterata, fino a raggiungere il caso base $f^{(k+1)}(n) \leq n_0$ per un determinato valore di $i = k$. Si definisce k come il massimo valore di i per cui la taglia dell'istanza è ancora maggiore di n_0 , ovvero:

$$k = f^*(n, n_0) := \max \left\{ k : f^{(k)}(n) > n_0 \right\}$$

Formulone

Il formulone è una formula generale che risolve l'equazione generale alle ricorrenze indicata sopra, se sono soddisfatte opportune condizioni indicate sotto.

$$T(n) = \underbrace{\sum_{l=0}^{f^*(n, n_0)} \left[\prod_{j=0}^{l-1} s(f^{(j)}(n)) \cdot \omega(f^{(l)}(n)) \right]}_{\text{lavoro compiuto nei nodi interni}} + \underbrace{\prod_{j=0}^{f^*(n, n_0)} s(f^{(j)}(n)) \cdot T_0(f^{(f^*(n, n_0)+1)}(n))}_{\text{lavoro compiuto nelle foglie}}$$

per ogni livello interno # nodi al livello l -esimo costo interno di ogni nodo al livello l -esimo # di foglie costo di ogni foglia

- il numero di figli dei nodi di un certo livello è costante, ovvero $s(f^{(l)}(n)) = s_l$ per ogni livello l
- la taglia di tutti i nodi di un certo livello è costante, ovvero $f^{(l)}(n) = f_l$ per ogni livello l

Formulone applicato ad un caso particolare

Di seguito un esempio di applicazione del formulone ad un caso particolare con:

- $s(n) = a$: numero di figli costante per ogni nodo interno
- $f(n) = n/2$: la taglia dell'istanza si dimezza ad ogni chiamata ricorsiva
- $\omega(n) = \beta n$: costo di divide and conquer lineare per ogni nodo interno
- $T_0(n) = 1$: costo costante per ogni foglia

$$\begin{aligned} T(n) &= \sum_{l=0}^{(\log_2 n)-1} \left(a^l \cdot \beta \frac{n}{2^l} \right) + a^{\log_2 n} \cdot T_0(n) = \beta n \cdot \sum_{l=0}^{(\log_2 n)-1} (a/2)^l + n^{\log_2 a} = \\ &= \beta n \cdot \frac{(a/2)^{\log_2 n} - 1}{(a/2) - 1} + n^{\log_2 a} = \frac{\beta}{a/2 - 1} \cdot n \left(n^{\log_2 a/2} - 1 \right) + n^{\log_2 a} = \\ &= \underbrace{\frac{\beta}{a/2 - 1} \cdot (n^{\log_2 a} - n)}_{\text{costo dei nodi interni}} + \underbrace{n^{\log_2 a}}_{\text{costo delle foglie}} = \Theta(n^{\log_2 a}) \end{aligned}$$

Complessità degli algoritmi di ordinamento basati sui confronti

Al momento non si conoscono funzioni che legano problemi alla relativa complessità asintotica e più in generale non si conoscono metodi per individuare in maniera assoluta i lower bound di problemi, ad eccezione di quelli deducibili banalmente.

Ad esempio gli algoritmi di ordinamento basati sui confronti hanno lower bound banale di $\Theta(n \log n)$, siccome $n \log n$ è la quantità di informazione necessaria a rappresentare tutte le possibili permutazioni di n elementi ed equivale al numero minimo di confronti (rappresentabili in bit) necessari per distinguere tutte le possibili permutazioni.

$$\# \text{permutazioni} = n! \approx n^n \quad \text{informazione}(\text{perm}_i) = \log_2(n!) \approx \log_2(n^n) = n \log_2 n$$

3.5 Master theorem

Assunzioni iniziali

Il master theorem è un teorema che fornisce una soluzione asintotica immediata in funzione di due parametri a e b per una classe di equazioni alle ricorrenze con le seguenti caratteristiche:

- $s(n) = a$: numero di figli costante per ogni nodo interno
- $f(n) = n/b$: la taglia dell'istanza si divide per una costante $b > 1$ ad ogni chiamata ricorsiva
- $n_0 = 1$: caso base per istanze di taglia 1

$$T(n) = \begin{cases} T_0(n) & \text{se } n \leq 1 \\ a \cdot T(n/b) + \omega(n) & \text{se } n > 1 \end{cases}$$

Funzione soglia

Si definisce una **funzione soglia** $\sigma(n)$ che rappresenta il numero di foglie dell'albero della ricorsione. Permette di calcolare in maniera immediata il costo complessivo dovuto al lavoro compiuto nelle foglie che funge da lower bound della complessità dell'algoritmo.

$$\sigma(n) = n^{\log_b a} \quad T(n) \geq T_{\text{foglie}}(n) = T_0(n) \cdot \# \text{foglie} = T_0(n) \cdot \sigma(n)$$

Classi di funzioni e relative complessità asintotiche

Paragonando il costo dovuto al lavoro compiuto da ogni nodo interno $\omega(n)$ con la funzione soglia $\sigma(n)$, si individuano tre casi per cui è possibile calcolare la complessità asintotica di $T(n)$ in maniera immediata:

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{se il lavoro nei nodi interni } \omega(n) \text{ è trascurabile rispetto a quello delle foglie } \sigma(n) \\ \Theta(n^{\log_b a} \log_b n) & \text{se il lavoro nei nodi interni } \omega(n) \text{ è dello stesso ordine di quello delle foglie } \sigma(n) \\ \Theta(\omega(n)) & \text{se il lavoro nei nodi interni } \omega(n) \text{ domina quello delle foglie } \sigma(n) \end{cases}$$

Analisi del caso 1

Si verifica quando il lavoro nei nodi interni $\omega(n)$ è trascurabile rispetto a quello delle foglie $\sigma(n)$:

$$\exists \varepsilon > 0 \text{ tale che } \omega(n) = O(n^{(\log_b a) - \varepsilon}) = \sigma(n) \cdot O(n^{-\varepsilon})$$

Di seguito si osserva che calcolando il contributo complessivo dei nodi interni usando $\omega(n) = n^{(\log_b a) - \varepsilon}$, il risultato è comparabile con quello delle foglie senza far aumentare la complessità asintotica di $\Theta(n^{\log_b a})$.

$$\begin{aligned} T_{\text{int}}(n) &= \sum_{l=0}^{(\log_b n)-1} a^l \cdot \omega\left(\frac{n}{b^l}\right) = \sum_{l=0}^{(\log_b n)-1} a^l \cdot \left(\frac{n}{b^l}\right)^{(\log_b a) - \varepsilon} = n^{\log_b a} n^{-\varepsilon} \cdot \sum_{l=0}^{(\log_b n)-1} \frac{a^l}{b^{l(\log_b a) - l\varepsilon}} = \\ &= n^{\log_b a} n^{-\varepsilon} \cdot \sum_{l=0}^{(\log_b n)-1} \frac{a^l}{a^l \cdot b^{l\varepsilon}} = n^{\log_b a} n^{-\varepsilon} \cdot \sum_{l=0}^{(\log_b n)-1} (b^\varepsilon)^l = n^{\log_b a} n^{-\varepsilon} \cdot \frac{(b^\varepsilon)^{(\log_b n)} - 1}{b^\varepsilon - 1} = \\ &= n^{\log_b a} n^{-\varepsilon} \cdot \frac{n^\varepsilon - 1}{b^\varepsilon - 1} \approx n^{\log_b a} n^{-\varepsilon} \cdot \frac{n^\varepsilon}{b^\varepsilon - 1} = \frac{n^{\log_b a}}{b^\varepsilon - 1} = \frac{\sigma(n)}{b^\varepsilon - 1} = O(\sigma(n)) \end{aligned}$$

Si osserva che per $\varepsilon \rightarrow 0^+$ il coefficiente moltiplicativo sarebbe molto elevato, ma comunque costante, per cui asintoticamente trascurabile.

Analisi del caso 2

Si verifica quando il lavoro nei nodi interni $\omega(n)$ è dello stesso ordine di quello delle foglie $\sigma(n)$:

$$\omega(n) = \Theta(n^{\log_b a}) = \Theta(\sigma(n))$$

Di seguito si dimostra che il contributo complessivo dei nodi interni ha una complessità asintotica maggiore di quella delle foglie e domina la complessità totale dell'algoritmo, che risulta essere $\Theta(n^{\log_b a} \log_b n)$.

$$\begin{aligned} T_{\text{int}}(n) &= \sum_{l=0}^{(\log_b n)-1} \left(a^l \cdot \omega\left(\frac{n}{b^l}\right) \right) = \sum_{l=0}^{(\log_b n)-1} \left(a^l \cdot \left(\frac{n}{b^l}\right)^{\log_b a} \right) = n^{\log_b a} \cdot \sum_{l=0}^{(\log_b n)-1} \left(\frac{a^l}{b^{l(\log_b a)}} \right) \\ &= n^{\log_b a} \cdot \sum_{l=0}^{(\log_b n)-1} \frac{a^l}{a^l} = n^{\log_b a} \cdot (\log_b n) = \Theta(n^{\log_b a} \log_b n) \end{aligned}$$

Di seguito si osserva che il costo complessivo di ogni livello ha la stessa complessità asintotica di quello delle foglie, per cui è possibile calcolare la complessità totale moltiplicando la complessità di ogni livello per il numero di livelli. Gli algoritmi di questa tipologia (es. mergesort e quicksort) hanno la particolarità che il costo delle fasi di divide o combine è proporzionale alla taglia dell'istanza.

$$T_{l\text{-esimo livello}}(n) = a^l \cdot \omega\left(\frac{n}{b^l}\right) = a^l \cdot \left(\frac{n}{b^l}\right)^{\log_b a} = a^l \cdot \frac{n^{\log_b a}}{b^{l(\log_b a)}} = a^l \cdot \frac{n^{\log_b a}}{a^l} = n^{\log_b a} = \sigma(n)$$

Analisi del caso 3

Si verifica quando il lavoro nei nodi interni $\omega(n)$ domina quello delle foglie $\sigma(n)$:

$$\exists \varepsilon > 0 \text{ tale che } \omega(n) = \Omega(n^{\log_b a + \varepsilon}) = \sigma(n) \cdot \Omega(n^\varepsilon)$$

Dalla condizione precedente si può dedurre anche la seguente condizione di regolarità, ovvero che il lavoro interno complessivo nei nodi figli (al livello inferiore) è minore del lavoro interno del nodo padre.

$$\exists c \in \mathbb{R} \text{ con } 0 < c < 1 \text{ tale che } \forall n \in \mathbb{N} \text{ si ha } a \cdot \omega\left(\frac{n}{b}\right) \leq c \cdot \omega(n)$$

Per la dimostrazione del terzo caso si utilizza la condizione di regolarità appena enunciata, in quanto rende lo svolgimento più semplice e immediato. Di seguito sono riportati alcuni passaggi preliminari che verranno poi usati nella successiva dimostrazione.

$$a \cdot \omega\left(\frac{n}{b}\right) \leq c \cdot \omega(n) \Rightarrow \omega\left(\frac{n}{b}\right) \leq \frac{c}{a} \cdot \omega(n) \Rightarrow \omega\left(\frac{n}{b^l}\right) \leq \left(\frac{c}{a}\right)^l \cdot \omega(n)$$

$$T_{\text{int}}(n) = \sum_{l=0}^{(\log_b n)-1} a^l \cdot \omega\left(\frac{n}{b^l}\right) \leq \sum_{l=0}^{(\log_b n)-1} a^l \cdot \frac{c^l}{a^l} \cdot \omega(n) = \omega(n) \cdot \sum_{l=0}^{(\log_b n)-1} c^l < \omega(n) \cdot \frac{1}{1-c} = \Theta(\omega(n))$$

Dall'analisi precedente si osserva che il costo complessivo dei nodi interni è proporzionale al costo di ogni nodo interno e domina la complessità totale dell'algoritmo se il costo del singolo nodo interno è asintoticamente maggiore di quello delle foglie, ovvero se $\omega(n) = \Omega(n^{\log_b a + \varepsilon})$.

Considerazione sugli altri casi

Esistono funzioni che non possono essere classificate in nessuno dei tre casi del master theorem, ad esempio quando $\omega(n) = n^{\log_b a} \log n$. In questi casi è necessario risolvere l'equazione alle ricorrenze attraverso altri metodi risolutivi, ad esempio il formulone o l'analisi analitica.

3.6 Applicazioni del divide and conquer

Nei seguenti capitoli sono riportati alcuni esempi di applicazioni efficienti del paradigma divide and conquer:

- esponenziazione con esponente intero
- moltiplicazione di polinomi tramite l'algoritmo di Karatsuba
- moltiplicazione di matrici tramite l'algoritmo di Strassen
- moltiplicazione di matrici supercircolanti tramite la trasformata di Hadamard
- moltiplicazione di polinomi tramite la trasformata di Fourier (FFT) e l'algoritmo di Cooley-Tukey

4 Esponenziazione con esponente intero

Formalizzazione del problema

Dato un elemento x proveniente da una struttura algebrica con operazione associativa (ad esempio numeri in $\mathbb{N}$, $\mathbb{R}$, $\mathbb{C}$, matrici, ...) costante, si vuole calcolare x^n con $n \in \mathbb{N}$. Siccome si tratta x come costante, si esclude che possa essere un polinomio per semplificare la risoluzione.

Osservazioni sulla taglia del problema

Essendo x costante, viene escluso dal calcolo della taglia che è determinata unicamente da n . Si osserva che la quantità di informazione necessaria a rappresentare n è $\log_2 n$, per cui la taglia del problema in funzione di cui va espressa la complessità è $\text{taglia}(n) = |n| = \log_2 n$, con $n = 2^{\text{taglia}(n)}$.

4.1 Algoritmo ovvio basato sulla definizione

L'algoritmo ovvio per calcolare x^n si basa sulla definizione ricorsiva di potenza come composizione multipla dell'operazione associativa della struttura algebrica, ovvero:

$$x^n := \begin{cases} x^{(0)} = 1 & \text{se } n = 0 \\ x^{(n-1)} \circ x & \text{se } n > 0 \end{cases} \quad - \quad \circ \text{ è l'operazione associativa della struttura algebrica}$$

La complessità equivale al costo di $n - 1$ composizioni dell'operazione associativa, per cui la complessità risulta essere esponenziale in funzione della taglia del problema, ovvero:

$$T(n) = n - 1 = 2^{\text{taglia}(n)} - 1 = \Theta\left(2^{\text{taglia}(n)}\right)$$

4.2 Algoritmo efficiente basato sul divide and conquer

Si osserva che se l'operazione è associativa, come richiesto nelle ipotesi, è possibile riarrangiare le composizioni di funzioni in modo da riutilizzare i risultati di composizioni già calcolate. Di seguito la definizione ricorsiva dell'algoritmo efficiente basato sul divide and conquer:

$$x^n := \begin{cases} x^{(0)} = 1 & \text{se } n = 0 \\ x^{(1)} = x & \text{se } n = 1 \\ x^{(n/2)} \circ x^{(n/2)} & \text{se } n > 1 \text{ è pari} \\ x^{\lfloor n/2 \rfloor} \circ x^{\lfloor n/2 \rfloor} \circ x & \text{se } n > 1 \text{ è dispari} \end{cases}$$

L'equazione alle ricorrenze per questo algoritmo al caso peggiore è la seguente, osservando che è sufficiente una chiamata ricorsiva per calcolare $x^{(n/2)}$ e, al caso peggiore, si aggiungono 2 operazioni di composizione.

$$T(n) = \begin{cases} 1 & \text{se } n = 0, 1 \\ T(n/2) + 2 & \text{se } n > 1 \end{cases}$$

Utilizzando il master theorem con $a = 1$, $b = 2$ e $\omega(n) = O(1)$, si ottiene una complessità lineare in funzione della taglia del problema, ovvero:

$$T(n) = \Theta(\log_2 n) = \Theta(\text{taglia}(n))$$

5 Moltiplicazione di polinomi - Karatsuba

Definizione formale di polinomio e degree bound

Un polinomio $P(x)$ di grado $N - 1$ (con N termini) è definito come:

$$P(x) := \sum_{i=0}^{N-1} a_i x^i \quad \text{con } a_0, a_1, \dots, a_{N-1} \text{ fissati}$$

Si osserva che un polinomio può essere visto anche come un numero espresso in base x con $0 \leq a_i < x$ secondo la notazione posizionale.

Il degree bound α è un limite superiore al grado di un polinomio, che indica l'insieme di tutti i polinomi di grado inferiore o uguale ad α , ovvero $N - 1 \leq \alpha$.

5.1 Prodotto tra polinomi usando la definizione

Tecnica risolutiva

Dati due polinomi $P(x)$ e $Q(x)$ di grado $N - 1$, il loro prodotto $P(x) \cdot Q(x)$ è definito come segue.

$$P(x) := \sum_{i=0}^{N-1} a_i x^i \quad Q(x) := \sum_{i=0}^{N-1} b_i x^i \quad P(x) \cdot Q(x) = \sum_{i=0}^{2N-2} c_i x^i$$

Analisi della complessità

Per calcolare il prodotto usando la definizione sono necessarie:

- N^2 moltiplicazioni tra i coefficienti a_i e b_j
- $(N - 1)^2 = N^2 - 1 - (2N - 2)$ addizioni tra i prodotti $a_i \cdot b_j$ per ottenere i coefficienti c_k

Complessivamente il prodotto usando la definizione ha complessità $\Theta(N^2)$.

5.2 Prodotto tra polinomi con approccio divide and conquer

Tecnica risolutiva

I polinomi $P(x)$ e $Q(x)$, di grado $N - 1$, possono essere scomposti in polinomi più semplici di grado $< N/2$ e il prodotto tra $P(x)$ e $Q(x)$ risulta essere in funzione del prodotto di questi polinomi più semplici, ovvero:

$$\begin{aligned} P(x) &= A(x) + B(x) \cdot x^{N/2} & Q(x) &= C(x) + D(x) \cdot x^{N/2} \\ P(x) \cdot Q(x) &= B(x) D(x) \cdot x^N + (A(x) D(x) + B(x) C(x)) \cdot x^{N/2} + A(x) C(x) \end{aligned}$$

Equazione alle ricorrenze

L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(N) = \begin{cases} 1 & \text{se } N = 1 \\ 4 \cdot T(N/2) + \beta_{std} N & \text{se } N > 1 \end{cases}$$

- il caso base è rappresentato dal prodotto di due polinomi di grado 0 che richiede un solo prodotto
- il passo ricorsivo prevede 4 chiamate ricorsive per calcolare i quattro prodotti AC , AD , BC e BD tra polinomi di grado $< N/2$
- il costo per ogni nodo interno è dato dalle somme dei coefficienti dei termini di grado uguale, che è proporzionale alla taglia N

Analisi della complessità

Dal formulone si ottiene che la complessità rimane sempre $\Theta(N^2)$, da come evidenziato di seguito:

$$\begin{aligned}
 T(N) &= \sum_{l=0}^{(\log_2 N)-1} \left(4^l \cdot \beta_{std} \frac{N}{2^l} \right) + 4^{\log_2 N} \cdot T_0(n) = \beta_{std} N \cdot \sum_{l=0}^{(\log_2 N)-1} (2^{2l}/2^l) + N^{\log_2 4} = \\
 &= \beta_{std} N \cdot \sum_{l=0}^{(\log_2 N)-1} 2^l + N^2 = \beta_{std} N \cdot \frac{2^{\log_2 N} - 1}{2 - 1} + N^2 = \\
 &= \underbrace{\beta_{std} (N^2 - N)}_{\text{costo delle somme nei nodi interni}} + \underbrace{N^2}_{\text{costo dei prodotti nelle foglie}} = \Theta(N^2)
 \end{aligned}$$

5.3 Prodotto di polinomi con l'algoritmo di Karatsuba (1962)

Tecnica risolutiva

Karatsuba osserva che è possibile ricavare la somma dei due prodotti $AD+BC$ riutilizzando i due prodotti AC e BD già precedentemente calcolati come segue:

$$(A + B) \cdot (C + D) = AC + (AD + BC) + BD \implies AD + BC = (A + B) \cdot (C + D) - AC - BD$$

In questo modo il numero di prodotti da calcolare ricorsivamente si riduce da 4 a 3, con l'effetto di aumentare lievemente il costo della fase di divide dovuto al calcolo delle somme $(A + B)$ e $(C + D)$.

Equazione alle ricorrenze ed analisi della complessità

L'equazione alle ricorrenze per l'algoritmo di Karatsuba è la seguente:

$$T(N) = \begin{cases} 1 & \text{se } N = 1 \\ 3 \cdot T(N/2) + \beta_{kz} N & \text{se } N > 1 \end{cases}$$

Dal formulone si ottiene che la complessità è $\Theta(N^{\log_2 3}) \approx \Theta(N^{1.58})$, da come evidenziato di seguito:

$$\begin{aligned}
 T(N) &= \sum_{l=0}^{(\log_2 N)-1} (3^l \cdot \beta_{kz} N/2^l) + 3^{\log_2 N} \cdot T(1) = \beta_{kz} N \cdot \sum_{l=0}^{(\log_2 N)-1} ((3/2)^l) + N^{\log_2 3} = \\
 &= \beta_{kz} N \cdot \frac{(3/2)^{\log_2 N} - 1}{(3/2) - 1} + N^{\log_2 3} = 2\beta_{kz} N \cdot (N^{\log_2 3 - \log_2 2} - 1) + N^{\log_2 3} = \\
 &= \underbrace{2\beta_{kz} (N^{\log_2 3} - N)}_{\text{costo delle somme nei nodi interni}} + \underbrace{N^{\log_2 3}}_{\text{costo dei prodotti nelle foglie}} = \Theta(N^{\log_2 3}) \approx \Theta(N^{1.58})
 \end{aligned}$$

Considerazioni sull'algoritmo di Karatsuba

- per valori di N piccoli, l'algoritmo standard (D&C) rimane ancora più efficiente di quello di Karatsuba, in quanto il costo aggiuntivo della fase di divide è maggiore del risparmio ottenuto dalla chiamata ricorsiva in meno
- per valori di N molto grandi, è possibile migliorare ulteriormente l'efficienza asintotica nel calcolo del prodotto di polinomi utilizzando ad esempio l'algoritmo di Toom-Cook
- la vera novità dell'algoritmo di Karatsuba è stata quella di trovare il primo algoritmo sub-quadratico per risolvere un problema che tutti i più grandi matematici ritenevano impossibile da risolvere in meno di $\Theta(N^2)$

6 Moltiplicazione di matrici - Strassen

Note introduttive sulle matrici

Per la seguente sezione relativa al prodotto di matrici, si considerano solo matrici quadrate $n \times n$ ($\in \mathbb{R}^{n \times n}$) con $n = 2^k$ potenza di 2, per un certo $k \in \mathbb{N}$. In questo modo le matrici sono perfettamente scomponibili ricorsivamente in quattro sottomatrici di dimensione $n/2 \times n/2$.

Per convenzione si assume che la taglia di una matrice è pari al numero di righe (o di colonne) di cui è composta, ovvero $A \in \mathbb{R}^{n \times n} \Rightarrow |A| = n$.

6.1 Prodotto di matrici usando la definizione

Tecnica risolutiva

Date due matrici $A, B \in \mathbb{R}^{n \times n}$, con $n = 2^k$, il loro prodotto $C = A \cdot B$ è sempre una matrice $C \in \mathbb{R}^{n \times n}$ i cui elementi c_{ij} sono calcolati come segue:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} \quad \text{per } i, j = 1, 2, \dots, n$$

In altre parole il prodotto di matrici è una matrice che ha come elementi il prodotto interno dei vettori riga e colonna rispettivamente della prima matrice A e della seconda matrice B .

Analisi della complessità

Il prodotto interno tra un vettore riga e un vettore colonna richiede n moltiplicazioni e $n - 1$ addizioni, per un costo totale di $T(n) = 2n - 1$. Siccome vanno fatti n^2 prodotti interni, il costo totale della moltiplicazione di matrici risulta $\Theta(n^3)$:

$$T(n) = n^2(2n - 1) = 2n^3 - n^2 = \Theta(n^3)$$

6.2 Prodotto di matrici con approccio divide and conquer

Tecnica risolutiva

Le matrici $A, B \in \mathbb{R}^{n \times n}$ possono essere scomposte ciascuna in quattro sottomatrici di dimensioni $n/2 \times n/2$, identificati con $X_{11}, X_{12}, X_{21}, X_{22} \in \mathbb{R}^{n/2 \times n/2}$. Il prodotto $C = A \cdot B$ risulta essere in funzione dei prodotti tra queste sottomatrici.

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} \\ C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} & C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{aligned}$$

Equazione alle ricorrenze

L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 8 \cdot T(n/2) + 4n^2 & \text{se } n > 1 \end{cases}$$

- il caso base è rappresentato dal prodotto di due matrici 1×1 che richiede un solo prodotto
- il passo ricorsivo prevede 8 chiamate ricorsive per calcolare gli otto prodotti tra le sottomatrici di dimensione $n/2 \times n/2$
- il costo per ogni nodo interno è dato dalle 4 somme dei prodotti tra sottomatrici $n/2 \times n/2$

Analisi della complessità

Dal formulone si ottiene che la complessità rimane sempre $\Theta(n^3)$, da come evidenziato di seguito:

$$\begin{aligned}
 T(n) &= \sum_{l=0}^{(\log_2 n)-1} \left(8^l \cdot \left(\frac{n}{2^l} \right)^2 \right) + 8^{\log_2 n} \cdot T_0(n) = n^2 \cdot \sum_{l=0}^{(\log_2 n)-1} 2^l + n^{\log_2 8} = n^2 \cdot \frac{2^{\log_2 n} - 1}{2 - 1} + n^3 = \\
 &= \underbrace{n^2 \cdot (n - 1)}_{\text{costo delle somme dei nodi interni}} + \underbrace{n^3}_{\text{costo dei prodotti alle foglie}} = 2n^3 - n^2 = \Theta(n^3)
 \end{aligned}$$

Vantaggi dell'algoritmo D&C ricorsivo rispetto a quello iterativo basato sulla definizione

Ad oggi le macchine moderne performano molto meglio con l'algoritmo D&C ricorsivo rispetto a quello iterativo basato sulla definizione in quanto si lavora con sottomatrici sempre più piccole fino a quando riescono ad essere totalmente memorizzate all'interno della cache. In questo modo si riducono drasticamente i tempi di accesso alla memoria principale dovuti ai continui cache miss quando si scorrono per intero righe e colonne per calcolare i prodotti interni.

6.3 Prodotto di matrici con l'algoritmo di Strassen (1969)

Tecnica risolutiva

Ispirandosi a Karatsuba, Strassen osserva che è possibile ricavare gli 8 prodotti tra le sottomatrici A_{ij} e B_{ij} riutilizzando solo 7 prodotti, calcolati su opportune somme e differenze di sottomatrici.

$$\begin{array}{lll}
 A_1 = (A_{12} - A_{22}) & B_1 = (B_{21} + B_{22}) & \\
 A_2 = (A_{11} + A_{22}) & B_2 = (B_{11} + B_{22}) & P_i = A_i \cdot B_i \quad \text{per } i = 1, 2, \dots, 7 \\
 A_3 = (A_{11} - A_{21}) & B_3 = (B_{11} + B_{12}) & C_{11} = P_1 + P_2 - P_4 + P_6 \\
 A_4 = (A_{11} + A_{12}) & B_4 = B_{22} & C_{12} = P_4 + P_5 \\
 A_5 = A_{11} & B_5 = (B_{12} - B_{22}) & C_{21} = P_6 + P_7 \\
 A_6 = A_{22} & B_6 = (B_{21} - B_{11}) & C_{22} = P_2 - P_3 + P_5 - P_7 \\
 A_7 = (A_{21} + A_{22}) & B_7 = B_{11} &
 \end{array}$$

Osservazioni sui prodotti di Strassen

- se si sviluppano i conti per le quattro espressioni di C_{ij} si può notare che si ottengono esattamente le stesse espressioni dell'approccio D&C standard
- esiste una simmetria tra le espressioni dei blocchi A_i e B_i : tra A_1 e B_5 , tra A_2 e B_2 , tra $-A_3$ e B_6 , tra A_4 e B_3 , tra A_5 e B_7 , tra A_6 e B_4 , tra A_7 e B_1

Equazione alle ricorrenze

L'equazione alle ricorrenze per l'algoritmo di Strassen è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 7 \cdot T(n/2) + 18 \cdot n^2/4 & \text{se } n > 1 \end{cases}$$

- il caso base è rappresentato dal prodotto di due matrici 1×1 che richiede un solo prodotto
- il passo ricorsivo prevede 7 chiamate ricorsive per calcolare i sette prodotti P_i tra i blocchi A_i e B_i di dimensione $n/2 \times n/2$
- il costo della fase di divide consiste nelle 10 somme e differenze tra sottomatrici $A_{ij}, B_{ij} \in \mathbb{R}^{n/2 \times n/2}$ per calcolare i blocchi A_i e B_i (complessivamente $10 \cdot n^2/4$)
- il costo della fase di conquer consiste nelle 8 somme/differenze tra i prodotti $P_i \in \mathbb{R}^{n/2 \times n/2}$ per calcolare i blocchi C_{ij} (complessivamente $8 \cdot n^2/4$)

Analisi della complessità

Applicando il formulone si ottiene che la complessità è $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$, da come mostrato di seguito:

$$\begin{aligned}
 T(n) &= \sum_{l=0}^{(\log_2 n)-1} \left(7^l \cdot \frac{18}{4} \cdot \left(\frac{n}{2^l} \right)^2 \right) + 7^{\log_2 n} \cdot T_0(n) = \frac{9}{2} n^2 \cdot \sum_{l=0}^{(\log_2 n)-1} ((7/4)^l) + n^{\log_2 7} = \\
 &= \frac{9}{2} n^2 \cdot \frac{(7/4)^{\log_2 n} - 1}{(7/4) - 1} + n^{\log_2 7} = \frac{9}{2} n^2 \cdot \frac{n^{\log_2(7/4)} - 1}{3/4} + n^{\log_2 7} = \\
 &= \frac{9}{2} \cdot \frac{4}{3} n^2 \cdot \left(n^{(\log_2 7)-2} - 1 \right) + n^{\log_2 7} = 6n^2 \cdot (n^{\log_2 7}/n^2 - 1) + n^{\log_2 7} \\
 &= \underbrace{6n^{\log_2 7} - 6n^2}_{\text{costo delle somme dei nodi interni}} + \underbrace{n^{\log_2 7}}_{\text{costo dei prodotti alle foglie}} = 7n^{\log_2 7} - 6n^2 = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})
 \end{aligned}$$

Osservazioni generali sull'algoritmo di Strassen

- una volta le moltiplicazioni di numeri erano molto costose, ora i blocchi logici sono altamente ottimizzati; inoltre molti processori dispongono dell'istruzione MADD (multiply and add) che consente di eseguire in un solo ciclo di clock una moltiplicazione seguita da un'addizione
- se si volesse eseguire il quadrato di una matrice A con l'algoritmo di Strassen, si risparmierebbero 5 somme e sottrazioni siccome $A_i = B_i \forall i$
- in generale non si può avere corrispondenza tra i blocchi A_i e B_i , perché se così fosse, l'algoritmo diventerebbe commutativo portando sicuramente ad un risultato sbagliato siccome la moltiplicazione di matrici non è commutativa
- utilizzando la definizione, la complessità asintotica del prodotto di matrici era n^Ω con $\Omega = 3$, con Strassen si è riusciti a ridurre il valore di Ω a $\log_2 7 \approx 2.81$; ad oggi si conoscono algoritmi per il prodotto di matrici con $\Omega \approx 2.37$, ma risultano poco vantaggiosi in quanto hanno costanti asintotiche molto elevate che li rendono svantaggiosi per valori di n piccoli

6.4 Algoritmo ibrido per il prodotto di matrici

Idea implementativa

Per valori di $n \leq n_0$ si osserva che l'algoritmo standard (D&C) rimane ancora più efficiente di quello di Strassen. Si vuole determinare tale valore n_0 e sviluppare un algoritmo tale che per $n > n_0$ utilizzi l'algoritmo di Strassen, mentre per $n < n_0$ utilizzi l'algoritmo standard (D&C).

Calcolo del parametro n_0

Per calcolare il parametro n_0 , si cerca il valore che minimizza la complessità dell'algoritmo di Strassen imponendo che per $n \leq n_0$ venga utilizzato l'algoritmo standard (D&C), ovvero:

- i livelli dell'albero della ricorsione arrivano a $(\log_2(n/n_0)) - 1$
- il costo delle foglie è pari alla complessità dell'algoritmo standard (D&C), ovvero $T_0(n) = 2n^3 - n^2$

$$\begin{aligned}
 T(n, n_0) &= \sum_{l=0}^{(\log_2(n/n_0))-1} \left(7^l \cdot \frac{18}{4} \cdot \left(\frac{n}{2^l} \right)^2 \right) + 7^{\log_2(n/n_0)} \cdot (2n_0^3 - n_0^2) = \dots = \\
 &= n^{\log_2 7} \cdot \underbrace{\frac{2n_0 + 5}{n_0^{(\log_2 7)-2}}}_{g(n_0)} - 6n^2
 \end{aligned}$$

Si procede a minimizzare la funzione $T(n, n_0)$ per trovare il valore ottimale di n_0 :

$$\min_{n_0} T(n, n_0) \longrightarrow \min_{n_0} g(n_0) \longrightarrow \begin{aligned} n_{0,\min} &\approx 10.477 \\ g(n_{0,\min}) &= 3.895 \end{aligned}$$

Considerazioni sul parametro n_0

Dal punto di vista teorico, si sceglie $n_0 = 8$ in quanto è la potenza di 2 più vicina a $n_{0,\min}$. Riscrivendo l'equazione alle ricorrenze e la complessità asintotica dell'algoritmo ibrido, si ottiene:

$$T(n, n_{0,\min}) = \begin{cases} 2n^3 - n^2 & \text{se } n \leq 8 \\ 7 \cdot T(n/2) + 18 n^2/4 & \text{se } n > 8 \end{cases} \quad T(n, n_{0,\min}) = \begin{cases} 2n^3 - n^2 & \text{se } n \leq 8 \\ 3.918 \cdot n^{\log_2 7} - 6n^2 & \text{se } n > 8 \end{cases}$$

Dal punto di vista pratico, non è detto che $n_0 = 8$ sia il valore ottimale, in quanto il modello teorico non tiene conto di overhead dovuti al cambio di algoritmo, alla gestione della cache e altri fattori specifici della macchina su cui viene eseguito l'algoritmo.

Ad oggi si sta sviluppando una branca di ricerca chiamata **adaptive software** che punta ad aggiustare dinamicamente i parametri degli algoritmi di algebra lineare (come n_0) attraverso analisi e test pratici effettuati sulla macchina stessa, al fine di ottenere le migliori prestazioni possibili per quella determinata macchina.

7 Moltiplicazione di matrici supercircolanti - Tr. di Hadamard

Definizione di matrici supercircolanti

Una matrice $A \in \mathbb{C}^{n \times n}$, con $n = 2^k$ si dice supercircolante se:

- per $n = 1$ (banalmente tutti i numeri sono considerati matrici supercircolanti))
- per $n > 1$ la matrice si può dividere in quattro blocchi $n/2 \times n/2$ uguali a due a due lungo la diagonale principale e la diagonale secondaria e tali blocchi sono a loro volta supercircolanti

$$A = \begin{bmatrix} A_1 & A_2 \\ A_2 & A_1 \end{bmatrix}$$

Osservazioni sulle matrici supercircolanti

- nelle matrici supercircolanti, ogni riga è ottenuta tramite uno shift circolare della riga precedente
- le righe successive alla prima sono univocamente definite a partire dalla prima riga, detta riga generatrice, tramite opportuni shift circolari
- dato che è sufficiente la prima riga per definire una matrice supercircolante, la taglia di una matrice supercircolante A è pari al numero di elementi della sua prima riga, ovvero n
- si osserva che l'insieme delle matrici supercircolanti è chiuso rispetto alla somma:

$$A + B = \begin{bmatrix} A_1 & A_2 \\ A_2 & A_1 \end{bmatrix} + \begin{bmatrix} B_1 & B_2 \\ B_2 & B_1 \end{bmatrix} = \begin{bmatrix} A_1 + B_1 & A_2 + B_2 \\ A_2 + B_2 & A_1 + B_1 \end{bmatrix} \quad \text{supercircolante per def.}$$

- si osserva che l'insieme delle matrici supercircolanti è chiuso rispetto alla moltiplicazione:

$$A \cdot B = \begin{bmatrix} A_1 & A_2 \\ A_2 & A_1 \end{bmatrix} \cdot \begin{bmatrix} B_1 & B_2 \\ B_2 & B_1 \end{bmatrix} = \begin{bmatrix} A_1 B_1 + A_2 B_2 & A_1 B_2 + A_2 B_1 \\ A_2 B_1 + A_1 B_2 & A_2 B_2 + A_1 B_1 \end{bmatrix} \quad \text{supercircolante per def.}$$

- siccome l'insieme delle matrici supercircolanti è chiuso rispetto alle operazioni di somma e moltiplicazione, allora è considerato un anello, in più si osserva che è anche commutativo, siccome le due operazioni godono della proprietà commutativa, ovvero $A + B = B + A$ e $A \cdot B = B \cdot A$

Somme tra matrici supercircolanti

Per sommare due matrici supercircolanti è sufficiente sommare le prime righe dei due addendi elemento per elemento, ottenendo la prima riga della matrice risultante dalla somma. La complessità di questo approccio è $T_{add}(n) = \Theta(n)$ con n pari alla taglia delle matrici.

7.1 Moltiplicazioni tra matrici supercircolanti

Moltiplicazioni usando la definizione

Usando la definizione di moltiplicazione di matrici, si osserva che si richiede il calcolo di 4 prodotti tra sottomatrici supercircolanti di dimensione $n/2 \times n/2$ e 2 somme tra matrici supercircolanti di dimensione $n/2 \times n/2$. L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 4 \cdot T(n/2) + 2 \cdot n/2 & \text{se } n > 1 \end{cases} = \begin{cases} 1 & \text{se } n = 1 \\ 4 \cdot T(n/2) + n & \text{se } n > 1 \end{cases}$$

Dal master theorem con $a = 4$, $b = 2$ e $\omega(n) = n$ si ottiene una funzione soglia di $\sigma(n) = n^{\log_2 4} = n^2$, tale per cui $\omega(n) = \sigma(n) \cdot O(n^{-1})$, ovvero si ricade nel primo caso del master theorem. La complessità è quindi $T(n) = \Theta(\sigma(n)) = \Theta(n^2)$.

Moltiplicazioni con approccio D&C ottimizzato

Si osserva che secondo opportune manipolazioni algebriche è possibile ricavare i due blocchi C_1 e C_2 del prodotto attraverso soli 2 prodotti, con l'overhead di 6 somme e due moltiplicazioni per uno scalare.

$$\begin{aligned} U &= (A_1 + A_2) \cdot (B_1 + B_2) = A_1 B_1 + A_1 B_2 + A_2 B_1 + A_2 B_2 & C_1 &= (U + V)/2 = A_1 B_1 + A_2 B_2 \\ V &= (A_1 - A_2) \cdot (B_1 - B_2) = A_1 B_1 - A_1 B_2 - A_2 B_1 + A_2 B_2 & C_2 &= (U - V)/2 = A_1 B_2 + A_2 B_1 \end{aligned}$$

L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2 \cdot T(n/2) + 6 \cdot n/2 + 2 \cdot n/2 & \text{se } n > 1 \end{cases} = \begin{cases} 1 & \text{se } n = 1 \\ 2 \cdot T(n/2) + 4n & \text{se } n > 1 \end{cases}$$

Dal master theorem con $a = 2$, $b = 2$ e $\omega(n) = 4n$ si ottiene una funzione soglia di $\sigma(n) = n^{\log_2 2} = n$ paragonabile a $\omega(n)$. La complessità dal secondo caso del master theorem risulta essere $T(n) = \Theta(n \cdot \log n)$.

7.2 Trasformata di Hadamard e Fast Hadamard Transform (FHT)

Autovalori alle foglie dell'albero della ricorsione

L'albero della ricorsione per il prodotto di matrici supercircolanti con approccio D&C ottimizzato è un albero binario completo in cui ogni nodo figlio sinistro riceve le somme tra i blocchi A_1 e A_2 e tra i blocchi B_1 e B_2 , mentre ogni nodo figlio destro riceve le differenze tra i medesimi blocchi, affinché siano moltiplicate opportunamente.

Le foglie ricevono due matrici degeneri in scalari a e b che è possibile dimostrare essere proprio gli autovalori delle due matrici supercircolanti A e B di partenza.

Trasformata e prodotto di Hadamard

È possibile interpretare le operazioni di divide come una trasformazione lineare che, partendo da una matrice supercircolante rappresentata con la sua prima riga, restituisce il vettore di autovalori della matrice supercircolante stessa. In questo modo il prodotto di matrici supercircolanti può essere interpretato come un prodotto di autovalori, che risulta avere complessità $\Theta(n)$ con n pari alla taglia delle matrici supercircolanti. Una volta fatto ciò, le operazioni di combine effettuano l'inverso della trasformazione lineare, restituendo la matrice supercircolante risultante dal prodotto.

La trasformazione lineare impiegata prende il nome di **trasformata di Hadamard**, mentre il prodotto membro a membro tra i vettori di autovalori prende il nome di **prodotto di Hadamard**.

La trasformata e la relativa antitrasformata di Hadamard possono essere calcolate in $\Theta(n \cdot \log n)$, mentre il prodotto di Hadamard può essere calcolato in $\Theta(n)$. Di conseguenza, il prodotto di matrici supercircolanti tramite la trasformata di Hadamard ha complessità complessiva di $\Theta(n \cdot \log n)$.

Cambio di rappresentazioni per risolvere problemi più efficientemente

L'idea alla base del prodotto di matrici supercircolanti tramite la trasformata di Hadamard è quello di convertire i dati in ingresso in una rappresentazione che permette di risolvere il problema richiesto in maniera più efficiente, per poi convertire il risultato ottenuto nella rappresentazione originale. Altri esempi di algoritmi che adottano questa tecnica sono:

- l'algoritmo di Strassen, anche se non è chiaro come interpretare la rappresentazione intermedia
- l'algoritmo di FFT, che utilizza una trasformata di Fourier per calcolare il prodotto di polinomi in maniera più efficiente

Osservazione sulle simmetrie

Una simmetria è una proprietà di invarianza rispetto ad una certa trasformazione. Ad esempio la proprietà commutativa è l'invarianza rispetto alla trasformazione che scambia i due operandi di un'operazione.

Conoscere le simmetrie permette di identificare ed eliminare i gradi di libertà di un problema, riducendo sia il peso della rappresentazione utilizzata che nelle operazioni di calcolo.

8 Moltiplicazione di polinomi e FFT - Fast Fourier Transform

Nota: la DFT (Discrete Fourier Transform) è la trasformata matematica discreta di Fourier, la FFT (Fast Fourier Transform) è l'algoritmo efficiente per calcolare la DFT.

Cenni sulla moltiplicazione di polinomi

Dati due polinomi $p(x)$ e $q(x)$ rispettivamente di degree bound n e m , il loro prodotto $p(x) \cdot q(x)$ è un polinomio di degree bound $n + m$ il cui vettore di coefficienti c è dato dal prodotto di convoluzione tra i vettori di coefficienti a e b dei polinomi $p(x)$ e $q(x)$, come illustrato di seguito:

$$p(x) = \sum_{i=0}^{n-1} a_i x^i = a_0 x^0 + a_1 x^1 + \dots + a_{n-1} x^{n-1} \quad q(x) = \sum_{i=0}^{m-1} b_i x^i = b_0 x^0 + b_1 x^1 + \dots + b_{m-1} x^{m-1}$$

$$p(x) \cdot q(x) = \sum_{i=0}^{n+m-2} c_i x^i \quad \text{con} \quad c_k = \sum_{\substack{i+j=k \\ 0 \leq i < n \\ 0 \leq j < m}} a_i b_j = \sum_{i=0}^k a_i \cdot b_{k-i} \quad \text{per } k = 0, 1, \dots, n+m$$

Applicando la pura definizione per il calcolo della convoluzione, la complessità risultante è quadratica nel degree bound dei polinomi. La soluzione più efficiente che verrà descritta di seguito consiste in:

- trasformare, tramite la DFT, i polinomi $p(x)$ e $q(x)$ dalla rappresentazione per vettori di coefficienti alla rappresentazione per vettori di punti o valori, valutando i polinomi nelle radici n-esime dell'unità, con complessità $\Theta(n \cdot \log n)$ con n pari al degree bound dei polinomi
- eseguire il prodotto di Hadamard tra i vettori di punti o valori ottenuti, che equivale al prodotto di convoluzione dei vettori di coefficienti, con complessità lineare nel degree bound dei polinomi con complessità $\Theta(n)$ con n pari al degree bound dei polinomi
- eseguire l'inversa della DFT per trasformare il vettore di punti o valori del polinomio risultante in un vettore di coefficienti con complessità $\Theta(n \cdot \log n)$ con n pari al degree bound dei polinomi

8.1 Rappresentazioni di un polinomio

Rappresentazione tramite vettore di coefficienti

Un polinomio $p(x)$ di degree bound n può essere rappresentato tramite un vettore di coefficienti $\underline{a}$ di dimensione n , in cui l'elemento a_i rappresenta il coefficiente del termine x^i del polinomio. Il prodotto di due polinomi rappresentati in questo modo è dato dal prodotto di convoluzione tra i vettori di coefficienti.

$$\underline{a} = (a_0, a_1, \dots, a_{n-1}) \quad \underline{b} = (b_0, b_1, \dots, b_{m-1}) \quad \underline{c} = (c_0, c_1, \dots, c_{n+m-2}) \quad \underline{c} = \underline{a} * \underline{b}$$

Rappresentazione tramite vettore di punti o valori

Un polinomio $p(x)$ di degree bound n può essere rappresentato tramite un vettore di punti o valori $\underline{y}$ di dimensione n , in cui l'elemento y_i rappresenta il valore del polinomio in un punto specifico x_i . Esistono infiniti modi di rappresentare un polinomio per punti, a seconda dei punti scelti, ma data una qualsiasi rappresentazione esiste solo un polinomio di grado n associato a tale rappresentazione.

Il prodotto di due polinomi rappresentati per punti è dato dal prodotto element-wise tra i vettori di punti, avendo l'accortezza di estenderli con nuovi elementi fino alla dimensione necessaria a rappresentare il polinomio risultante, ovvero $n + m - 1$ se i polinomi di partenza hanno degree bound n e m .

$$(\underline{x}, \underline{y}) \text{ rappresentazione per punti di } p(x) \text{ con } \begin{matrix} \underline{x} = (x_0, x_1, \dots, x_{n-1}) \\ \underline{y} = (y_0, y_1, \dots, y_{n-1}) \end{matrix} \text{ e } y_i = p(x_i)$$

8.2 Valutazione di un polinomio

Valutazione di un polinomio usando la definizione

Il processo di valutazione di un polinomio $p(x)$ consiste nel calcolare il valore del polinomio in un punto specifico x_i . Tale processo permette di trasformare un polinomio dalla rappresentazione per vettore di coefficienti alla rappresentazione per punti.

Il costo di valutazione di un polinomio $p(x)$ di degree bound n in un punto x_i usando la definizione di polinomio è lineare nel degree bound del polinomio.

$$T(n) = \underbrace{n-2}_{\text{potenze } x^2, x^3, \dots} + \underbrace{n-1}_{\text{prodotti } a_i \cdot x^i} + \underbrace{n+1}_{\text{addizioni}} = 3n - 4 = \Theta(n)$$

Considerando che un polinomio deve essere valutato in n punti per essere trasformato in una rappresentazione per punti, la complessità complessiva della trasformazione risulta quadratica.

Valutazione di un polinomio con l'algoritmo di Horner

L'idea dell'algoritmo di Horner è quella di riscrivere un polinomio $p(x)$ in una forma che permette di valutare il polinomio ricorsivamente come segue:

$$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1} = a_0 + x \cdot (a_1 + a_2x + \dots + a_{n-1}x^{n-2})$$

L'equazione alle ricorrenze e la complessità dell'algoritmo di Horner rimangono lineari nel degree bound del polinomio, per cui non c'è un netto miglioramento rispetto all'approccio precedente.

$$T(n) = \begin{cases} 0 & \text{se } n = 1 \\ T(n-1) + 2 & \text{se } n > 1 \end{cases} \quad T(n) = 2(n-1) = \Theta(n)$$

Valutazione di un polinomio sulle radici dell'unità

Questo approccio si basa sulla possibilità di riutilizzare i conti fatti per valutare un polinomio $p(x)$ in un punto x_i per valutare lo stesso polinomio in altri punti x_j , legati a x_i da una certa simmetria. Di seguito viene illustrato un esempio per un polinomio di degree bound $n = 8$.

$p(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 + a_6x^6 + a_7x^7$	polinomio $p(x)$ di degree bound 8
$p(x) = \underbrace{(a_0 + a_2x^2 + a_4x^4 + a_6x^6)}_{p^{[0]}(x^2)} + x \cdot \underbrace{(a_1 + a_3x^2 + a_5x^4 + a_7x^6)}_{p^{[1]}(x^2)}$	si dividono le potenze pari e dispari ottenendo due polinomi in x^2 di degree bound 4
$p(x) = p^{[0]}(x^2) + x \cdot p^{[1]}(x^2) \quad p(-x) = p^{[0]}((-x)^2) - x \cdot p^{[1]}((-x)^2) = p^{[0]}(x^2) - x \cdot p^{[1]}(x^2)$	le valutazioni del polinomio in $p(x)$ e $p(-x)$ condividono gli stessi due contributi $p^{[0]}(x^2)$ e $p^{[1]}(x^2)$
$p^{[0]}(x^2) = p^{[0][0]}(x^4) + x^2 \cdot p^{[0][1]}(x^4) \quad p^{[1]}(x^2) = p^{[1][0]}(x^4) + x^2 \cdot p^{[1][1]}(x^4)$ $p^{[0]}(-x^2) = p^{[0][0]}(x^4) - x^2 \cdot p^{[0][1]}(x^4) \quad p^{[1]}(-x^2) = p^{[1][0]}(x^4) - x^2 \cdot p^{[1][1]}(x^4)$	calcolando i polinomi $p^{[0]}(x^2)$ e $p^{[1]}(x^2)$ si ottengono "gratis" $p^{[0]}(-x^2)$ e $p^{[1]}(-x^2)$ che forniscono anche le due valutazioni $p(ix)$ e $p(-ix)$ con $i^2 = -1$
$p(ix) = p^{[0]}(ix^2) + ix \cdot p^{[1]}(ix^2) \quad p(ix) = p^{[0]}(ix^2) - ix \cdot p^{[1]}(ix^2)$ $= p^{[0]}(-x^2) + ix \cdot p^{[1]}(-x^2) \quad = p^{[0]}(-x^2) - ix \cdot p^{[1]}(-x^2)$	

È possibile infine eseguire un'ulteriore divisione ottenendo gli otto contributi $p^{[i][j][k]}(x^8)$, ciascuno di degree bound $8/2/2/2 = 1$ che forniscono le valutazioni del polinomio $p(x)$ anche per $x \pm ix$ e $-x \pm ix$. Si valuta, $p(x)$ sugli 8 punti $\pm x, \pm ix, x \pm ix, -x \pm ix$ che sono proprio le radici ottave dell'unità.

Successivamente si dimostrerà che questo approccio (basato sulla DFT) ha complessità $\Theta(n \cdot \log n)$ con n pari al degree bound del polinomio, contro la complessità quadratica degli approcci precedenti.

8.3 Interpolazione di un polinomio

Interpolazione con la matrice di Vandermonde

L'interpolazione di un polinomio $p(x)$ consiste nel calcolare il vettore di coefficienti $\underline{a}$ a partire dal vettore di punti $\underline{y}$. Tale processo permette di trasformare un polinomio dalla rappresentazione per punti alla rappresentazione per vettore di coefficienti.

Un possibile approccio per l'interpolazione consiste nell'invertire la formula di valutazione di un polinomio di grado n , ottenendo così un sistema a n equazioni con n incognite $(a_0, a_1, \dots, a_{n-1})$, i cui coefficienti sono le potenze di x .

$$p(x) = y = \sum_{i=0}^{n-1} a_i x^i \rightarrow \sum_{i=0}^{n-1} x_j^i a_i = y_j \rightarrow \underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{n-1} \end{bmatrix}}_{\text{matrice di Vandermonde } V(x_0, x_1, \dots, x_{n-1})} \cdot \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

La matrice delle potenze di x è detta matrice di Vandermonde, denotata con $V(x_0, x_1, \dots, x_{n-1})$. Il determinante della matrice di Vandermonde è dato dalla seguente formula.

$$\det(V(\underline{x})) = \prod_{0 \leq i < j \leq n-1} (x_j - x_i)$$

Se i punti $x_0, x_1, \dots, x_{n-1}$ sono distinti, allora il determinante della matrice di Vandermonde è sempre diverso da zero. Questo implica che la matrice è invertibile e il sistema di equazioni ha una soluzione unica. La complessità rimane quadratica nel degree bound del polinomio dovuta al prodotto tra matrice inversa e vettore di punti, supponendo di disporre dell'inversa della matrice di Vandermonde gratis.

$$\underline{a} = [V(x_0, x_1, \dots, x_{n-1})]^{-1} \cdot \underline{y}$$

Interpolazione con il polinomio interpolatore di Lagrange

Un altro approccio per l'interpolazione consiste nell'utilizzare il polinomio interpolatore di Lagrange

$$p(x) = \sum_{k=0}^{n-1} y_k L_k(x) \quad L_k(x) = \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}$$

Si osserva che quando $x = x_k$, significa che si sta cercando di valutare il polinomio $L_k(x)$ in un punto x_k , il polinomio interpolatore $L_k(x) = 1$ solo per k , mentre per gli altri $L_i(x) = 0$ e il risultato della sommatoria è proprio y_k .

Il costo computazionale dell'approccio con il polinomio interpolatore di Lagrange rimane sempre quadratico nel degree bound del polinomio da interpolare, in quanto è necessario calcolare n polinomi interpolatori di Lagrange $L_k(x)$ (per $k \in [0, n-1]$), ciascuno di grado $n-1$.

Interpolazione di un polinomio sulle radici dell'unità

Nei prossimi capitoli si osserverà che la valutazione di un polinomio $p(x)$ sulle radici n -esime dell'unità equivale al prodotto tra la matrice di Fourier e il vettore dei coefficienti del polinomio $p(x)$. Di conseguenza, per la trasformata inversa si utilizza l'inversa della matrice di Fourier e il vettore dei valori del polinomio $p(x)$ sulle radici n -esime dell'unità.

La valutazione costa $\Theta(n \log_2 n)$, si dimostrerà che anche l'interpolazione costa $\Theta(n \log_2 n)$, ottenendo così un sistema efficiente per cambiare rappresentazione e ottenere un algoritmo efficiente per il prodotto di polinomi rappresentati per punti.

8.4 Matrice di Fourier e radici n-esime dell'unità

Definizione di matrice di Fourier

La matrice di Fourier $F_n \in \mathbb{C}^{n \times n}$ è definita come la matrice di Vandermonde V i cui coefficienti sono le radici n -esime dell'unità.

$$F_n = V(\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}) \quad \text{con } \omega_n = e^{-2\pi i/n} \text{ radice principale dell'unità}$$

Radici n-esime dell'unità con relative proprietà e lemmi

- una radice n -esima dell'unità è un numero complesso $z_k = e^{i\varphi}$ con $\varphi = 2\pi k/n$ e $k \in [0, n-1]$

$$z_k^n = [\exp(2\pi i k/n)]^n = \exp(2\pi i k/n \cdot n) = \exp(2\pi i k) = 1$$

- una radice z_k è detta radice principale ω_n se elevata per i vari valori di $k \in [0, n-1]$ genera tutte le restanti radici n -esime; esistono più radici principali, solitamente si sceglie quella con $k = 1$

$$\omega_n^k = z_1^k = [\exp(2\pi i/n)]^k = \exp(2\pi i k/n) = z_k$$

$$\omega_n^i \cdot \omega_n^j = \omega_n^{i+j}$$

chiusura della moltiplicazione

l'insieme delle radici n -esime dell'unità
è chiuso rispetto alla moltiplicazione

$$\omega_n^k \cdot \omega_n^{-k} = 1$$

elemento inverso

l'insieme delle radici n -esime dell'unità possiede
l'elemento inverso ω_n^{n-k} rispetto alla moltiplicazione

$$\omega_{d \cdot n}^{d \cdot k} = \omega_n^k$$

lemma di cancellazione

moltiplicando per uno stesso fattore d sia l'esponente
che il grado della radice, il risultato non cambia

$$\omega_n^{n/2} = \omega_{2 \cdot n/2}^{1 \cdot n/2} = \omega_2$$

corollario del lemma di cancellazione

$$(\omega_n^i)^2 = (\omega_n^{i+n/2})^2 = \omega_n^i$$

lemma di dimezzamento

le due radici ω_n^i e $\omega_n^{i+n/2}$ sono opposte
ed i loro quadrati sono uguali

$$\sum_{j=0}^{n-1} (\omega_n^i)^j = \begin{cases} 0 & \text{se } i \bmod n = 0 \\ n & \text{altrimenti} \end{cases}$$

lemma di somma

la somma di tutte le potenze di una radice n -esima
dell'unità è zero se la radice è diversa da 1, altrimenti è n

NOTA: il lemma di somma indica anche la somma degli elementi lungo la righe o la colonna i -esima della matrice di Fourier, per cui la somma di tutte le righe e di tutte le colonne della matrice di Fourier è zero, tranne per la prima riga e la prima colonna, la cui somma è n .

Inversa della matrice di Fourier

La matrice di Fourier è invertibile essendo una Vandermonde con determinante pari a n (per il lemma di somma) in quanto è composta dalle radici n -esime dell'unità, tutte distinte tra loro.

$$F_{\omega_n}(i, j) = \omega_n^{ij} \quad F_{\omega_n}^{-1}(i, j) = \frac{1}{n} \cdot \omega_n^{-ij} = \frac{1}{n} \cdot [F_{\omega_n}(i, j)]^{-1} = \frac{1}{n} \cdot F_{\omega_n}(n-i, j)$$

$$\begin{aligned} F_n^{-1} &= \frac{1}{n} \cdot V\left((\omega_n^{-1})^0, (\omega_n^{-1})^1, (\omega_n^{-1})^2, \dots, (\omega_n^{-1})^{n-1}\right) \\ &= \frac{1}{n} \cdot V\left((\omega_n^0)^{n-1}, (\omega_n^1)^{n-1}, (\omega_n^2)^{n-1}, \dots, (\omega_n^{n-1})^{n-1}\right) \\ &= \frac{1}{n} \cdot V(\omega_n^0, \omega_n^{n-1}, \omega_n^{n-2}, \dots, \omega_n^1) \end{aligned}$$

La matrice inversa di Fourier è la matrice di Fourier scalata di $1/n$ in cui le righe da 1 a $n-1$ vengono invertite e l'insieme dei generatori è l'ultima riga della matrice di partenza.

$$(F_n^{-1} \cdot F_n)(h, k) = \sum_{l=0}^{n-1} F_n^{-1}(h, l) \cdot F_n(l, k) = \sum_{l=0}^{n-1} \frac{1}{n} \omega_n^{-hl} \omega_n^{lk} = \frac{1}{n} \sum_{l=0}^{n-1} \omega_n^{l(k-h)} = \begin{cases} 1 & \text{se } h=k \\ 0 & \text{altrimenti} \end{cases} = \mathbb{I}_n(h, k)$$

8.5 Discrete Fourier Transform e algoritmo FFT

Valutazione del polinomio sulle radici dell'unità con la matrice di Fourier

Si osserva che il processo di valutazione di un polinomio $p(x)$ sulle radici n -esime dell'unità, equivale al prodotto tra la matrice di Fourier F_n e il vettore dei coefficienti $\underline{a}$ del polinomio $p(x)$. Come per la matrice di Vandermonde, però, la complessità del prodotto applicando la pura definizione è $\Theta(n^2)$.

$$\underline{y} = \begin{pmatrix} p(\omega_0) \\ p(\omega_1) \\ \vdots \\ p(\omega_{n-1}) \end{pmatrix} = \begin{pmatrix} \sum_{i=0}^{n-1} a_i \omega_0^i \\ \sum_{i=0}^{n-1} a_i \omega_1^i \\ \vdots \\ \sum_{i=0}^{n-1} a_i \omega_{n-1}^i \end{pmatrix} = \begin{pmatrix} \omega_0^0 & \omega_0^1 & \dots & \omega_0^{n-1} \\ \omega_1^0 & \omega_1^1 & \dots & \omega_1^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ \omega_{n-1}^0 & \omega_{n-1}^1 & \dots & \omega_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = F_n \cdot \underline{a}$$

Formalizzazione della DFT

Il prodotto tra la matrice di Fourier F_n e un generico vettore $\underline{a}$ viene chiamato Discrete Fourier Transform (DFT) del vettore $\underline{a}$. È possibile, quindi, definire tale prodotto come una funzione come segue:

$$\underline{y} = F_n \cdot \underline{a} = \text{DFT}_n(\underline{a})$$

Algoritmo efficiente per calcolare la DFT

Si riprende la proprietà di sottostruttura individuata per valutare un polinomio $p(x)$ per due generici punti x e $-x$ e la si applica per due generiche radici n -esime dell'unità opposte tra loro ($\pm \omega_n^i$).

$$\begin{cases} p(x) = p^{[0]}(x^2) + x \cdot p^{[1]}(x^2) \\ p(-x) = p^{[0]}(x^2) - x \cdot p^{[1]}(x^2) \end{cases} \longrightarrow \begin{cases} p(\omega_n^i) = p^{[0]}(\omega_{n/2}^i) + \omega_n^i \cdot p^{[1]}(\omega_{n/2}^i) \\ p(\omega_n^{i+n/2}) = p^{[0]}(\omega_{n/2}^i) - \omega_n^i \cdot p^{[1]}(\omega_{n/2}^i) \end{cases}$$

$$\text{osservando che } -\omega_n^i = \omega_n^{i+n/2} \quad (\omega_n^i)^2 = (\omega_n^{i+n/2})^2 = \omega_{n/2}^i$$

Si costruisce il vettore dei valori $\underline{y}$ che assume il polinomio valutato sulle radici n -esime dell'unità:

$$\underline{y} = \text{DFT}_n(\underline{a}) = \begin{cases} y_i = y_i^{[0]} + \omega_n^i \cdot y_i^{[1]} \\ y_{i+n/2} = y_i^{[0]} - \omega_n^i \cdot y_i^{[1]} \end{cases} \quad \text{per } i \in [0, n/2-1] \quad \text{con} \quad \begin{cases} y_i^{[0]} = p^{[0]}(\omega_{n/2}^i) \\ y_i^{[1]} = p^{[1]}(\omega_{n/2}^i) \end{cases}$$

Si osserva che i vettori $\underline{y}^{[0]}$ e $\underline{y}^{[1]}$ corrispondono alla valutazione dei polinomi $p^{[0]}(x)$ e $p^{[1]}(x)$ sulle radici $n/2$ -esime dell'unità, che possono essere calcolati riapplicando lo stesso approccio in maniera ricorsiva.

$$y^{[0]} = F_{n/2} \cdot \underline{a}^{[0]} = \text{DFT}_{n/2}(\underline{a}^{[0]}) \quad \text{e} \quad y^{[1]} = F_{n/2} \cdot \underline{a}^{[1]} = \text{DFT}_{n/2}(\underline{a}^{[1]})$$

Analisi della complessità dell'algoritmo efficiente per calcolare la DFT

Il costo di ogni nodo interno è dato interamente dal costo del combine che comprende:

- $n/2$ moltiplicazioni tra i valori $y_i^{[1]}$ e le radici n -esime dell'unità ω_n^i
- $n/2$ addizioni per calcolare i valori y_i
- $n/2$ sottrazioni per calcolare i valori $y_{i+n/2}$
- $n/2 - 2$ prodotti per calcolare le radici n -esime dell'unità ω_n^i a partire da ω_n ; il -2 è dovuto al fatto che $\omega_n^0 = 1$ e $\omega_n^1 = \omega_n$ sono banali e non serve calcolarle

L'equazione alle ricorrenze e la complessità asintotica risultano essere:

$$T(n) = \begin{cases} 0 & \text{se } n = 1 \\ 2 \cdot T(n/2) + 2n - 2 & \text{se } n > 1 \end{cases} \quad T(n) = 2n \log_2 n - 2 \log_2 n$$

Osservazione dei valori alle foglie dell'albero della ricorsione

Costruendo l'albero della ricorsione si può osservare che i valori alle foglie sono ordinati secondo la permutazione bit-reversal. Tale permutazione è ottenuta invertendo i bit dell'indice i che identifica la foglia (es. $i = 6$ con rappresentazione binaria 110 diventa 011 che corrisponde a 3).

$$\begin{array}{ll}
 \text{DFT}_n(\underline{a}) & (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7) \\
 \text{DFT}_{n/2}(\underline{a}^{[*]}) & (a_0, a_2, a_4, a_6) \quad (a_1, a_3, a_5, a_7) \\
 \text{DFT}_{n/4}(\underline{a}^{[*][*]}) & (a_0, a_4) \quad (a_2, a_6) \quad (a_1, a_5) \quad (a_3, a_7) \\
 \text{DFT}_{n/8}(\underline{a}^{[*][*][*]}) & (a_0) \quad (a_4) \quad (a_2) \quad (a_6) \quad (a_1) \quad (a_5) \quad (a_3) \quad (a_7)
 \end{array}$$

Implementazione in pseudocodice dell'algoritmo FFT

0: **Algoritmo:** FFT($\underline{a}$)

Input: rappresentazione tramite vettore di coefficienti $\underline{a}$ del polinomio $p(x)$

Output: rappresentazione tramite vettore di punti $\underline{y}$ del polinomio $p(x)$

```

1:   $n \leftarrow \text{length}(\underline{a})$ 
2:
3:  if  $n = 1$  then                                     ▷ caso base
4:    return  $a$ 
5:
6:   $\underline{a}^{[0]} \leftarrow (a_0, a_2, a_4, \dots, a_{n-2})$           ▷ fase di divide
7:   $\underline{a}^{[1]} \leftarrow (a_1, a_3, a_5, \dots, a_{n-1})$ 
8:
9:   $\underline{y}^{[0]} \leftarrow \text{FFT}(\underline{a}^{[0]})$                           ▷ chiamate ricorsive
10:  $\underline{y}^{[1]} \leftarrow \text{FFT}(\underline{a}^{[1]})$ 
11:
12:   $\omega_n \leftarrow \exp(2\pi i/n)$                              ▷ fase di combine
13:   $\omega_i \leftarrow 1$ 
14:  for  $i \leftarrow 0$  to  $n/2 - 1$  do
15:     $t \leftarrow \omega_i \cdot y_i^{[1]}$ 
16:     $y_i \leftarrow y_i^{[0]} + t$ 
17:     $y_{i+n/2} \leftarrow y_i^{[0]} - t$ 
18:     $\omega_i \leftarrow \omega_i \cdot \omega_n$ 
19:  return  $\underline{y}$ 

```

8.6 Moltiplicazione di polinomi con la DFT e padding dei coefficienti

Padding dei coefficienti per il prodotto di polinomi

Il prodotto di due polinomi $p(x)$ e $q(x)$ con degree bound rispettivamente n e m è un polinomio $r(x) = p(x) \cdot q(x)$ con degree bound $n + m - 1$. Per poter rappresentare correttamente tramite vettore di punti il polinomio risultante è necessario valutare $r(x)$ su almeno $n + m - 1$ punti distinti.

Dal prodotto element-wise tra i vettori di punti di $p(x)$ e $q(x)$ si otterrà un vettore di punti non sufficientemente lungo per rappresentare $r(x)$. È necessario estendere le rappresentazioni tramite vettore di coefficienti al degree bound $n + m - 1$ del polinomio risultante, aggiungendo elementi nulli ai vettori di coefficienti di $p(x)$ e $q(x)$. Questa operazione è detta padding dei coefficienti.

$$\begin{aligned}\underline{a} = (a_0, a_1, \dots, a_{n-1}) &\rightarrow \underline{a} = (a_0, a_1, \dots, a_{n-1}, \overbrace{0, 0, \dots, 0}^{m \text{ elementi}}) \\ \underline{b} = (b_0, b_1, \dots, b_{m-1}) &\rightarrow \underline{b} = (b_0, b_1, \dots, b_{m-1}, \underbrace{0, 0, \dots, 0}_{n \text{ elementi}})\end{aligned}$$

Padding per raggiungere la potenza di due più vicina

Siccome l'algoritmo FFT è stato strutturato per funzionare con polinomi di degree bound pari ad una potenza di due, è necessario estendere i vettori di coefficienti al degree bound pari alla potenza di due maggiore più vicina al degree bound del polinomio risultante. Sarà necessario aggiungere ulteriori elementi nulli di padding ai vettori di coefficienti.

$$\begin{aligned}\underline{a} = (a_0, a_1, \dots, a_{n-1}, 0, 0, \dots, 0) &\rightarrow \underline{a} = (\overbrace{a_0, a_1, \dots, a_{n-1}, 0, 0, \dots, 0}^{2^k > n+m-1 \text{ elementi}}) \\ \underline{b} = (b_0, b_1, \dots, b_{m-1}, 0, 0, \dots, 0) &\rightarrow \underline{b} = (\overbrace{b_0, b_1, \dots, b_{m-1}, 0, 0, \dots, 0}^{2^k > n+m-1 \text{ elementi}})\end{aligned}$$

Note sulla complessità dopo il padding

Definita $N = 2^k$ la dimensione dei vettori di coefficienti dopo i due padding, la complessità del prodotto di polinomi tramite la DFT è $\Theta(N \log_2 N)$.

Supponendo che i degree bound dei polinomi $p(x)$ e $q(x)$ siano entrambi n , il primo padding porterà la dimensione dei vettori a $2n$, pari al degree bound del polinomio risultante. La potenza più vicina a $2n$ potrà essere al massimo $2 \cdot 2n = 4n$, siccome due potenze di due consecutive differiscono di un fattore 2.

Di conseguenza, la dimensione finale sarà al massimo $N = 4n$, che provocherà un peggioramento di circa quattro volte della complessità del prodotto, ma senza alterare quella asintotica di $\Theta(n \log_2 n)$.

Note sull'interpretazione del padding in generale

In generale non sempre è possibile applicare un padding all'input per ottenere una taglia di dimensione ottimale. Di seguito alcune osservazioni utili per interpretare il padding in generale:

- nel prodotto tra polinomi, il padding di elementi nulli non altera il polinomio, in quanto equivale ad aggiungere coefficienti di grado superiore nulli
- nel mergesort, il padding potrebbe essere fatto con valori facili da discriminare (es. $-\infty$ o $+\infty$) per cui dopo averli eliminati dal risultato, questo rimane sempre corretto
- nella trasformata di Fourier, gli elementi da aggiungere al vettore di coefficienti sono molto complessi da determinare, per cui non è possibile applicare un padding in maniera efficiente

NOTA: il padding di zeri nei vettori di input della DFT per la moltiplicazione di polinomi altera il risultato della DFT perché banalmente si sta valutando il polinomio su radici dell'unità in numero superiore. Tuttavia, in questo contesto non è un problema, in quanto entrambe le rappresentazioni (seppur sostanzialmente diverse) sono equivalenti per rappresentare lo stesso polinomio di partenza.

Padding per k -potenza di un polinomio

Si osserva che man mano che il polinomio viene elevato a potenza, il degree bound continua a crescere e sarebbe necessario effettuare continui padding per mantenere la dimensione dei vettori di coefficienti sufficientemente grandi per rappresentare i risultati intermedi e pari ad una potenza di due.

In alternativa a tale processo di padding continuo, si può effettuare un padding iniziale già sufficientemente grande per garantire che tutti i risultati intermedi e finali siano rappresentabili senza dover effettuare ulteriori padding.

Analizzando la complessità di entrambi i processi, si osserva che il costo complessivo dell'elevamento a potenza è praticamente lo stesso ed è pari a $\Theta(2kn \log_2 kn)$, ma che non altera la complessità asintotica.

8.7 Osservazioni utili per l'esame

Vettore di output di una trasformazione data la matrice e un certo vettore di input

Quando viene richiesto di calcolare il vettore di output di una trasformazione data la matrice relativa e un certo vettore di input, è utile osservare che il vettore di output è dato dalla combinazione lineare delle colonne della matrice pesate dai coefficienti del vettore di input.

$$\underline{y} = F \cdot \underline{a} = \sum_{i=0}^{n-1} F(*, i) \cdot a_i \quad \text{con } F(*, i) \text{ colonna } i\text{-esima di } F$$

Nel caso della trasformata di Fourier esistono alcuni vettori notevoli:

- $\underline{a} = [1, 0, 0, \dots, 0] \rightarrow \underline{y} = F_n \cdot \underline{a} = [1, 1, 1, \dots, 1]$ siccome la prima colonna della matrice di Fourier è composta da tutti 1
- $\underline{a} = [0, 1, 0, 0, \dots, 0] \rightarrow \underline{y} = F_n \cdot \underline{a} = [\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}]$ siccome la seconda colonna della matrice di Fourier è composta dalle radici n -esime dell'unità
- $\underline{a} = [1, 1, 1, \dots, 1] \rightarrow \underline{y} = F_n \cdot \underline{a} = [n, 0, 0, \dots, 0]$ siccome la somma di tutte le righe della matrice di Fourier è zero, tranne per la prima riga, la cui somma è n (lemma di somma)

Composizione di trasformate di Fourier

La composizione di due trasformate di Fourier F_n^2 provoca un ribaltamento degli elementi del vettore di input, escluso il primo elemento, che rimane invariato, a cui si aggiunge uno scalamento di n .

$$F_n^2 \cdot \underline{a} = (a_0, a_{n-1}, a_{n-2}, \dots, a_1) \cdot n$$

Siccome il ribaltamento applicato due volte riporta gli elementi del vettore di input alla posizione originale, si ha che la composizione di quattro trasformate di Fourier F_n^4 provoca soltanto uno scalamento di n^2 senza modificare la posizione degli elementi del vettore di input.

$$F_n^4 \cdot \underline{a} = (a_0, a_1, a_2, \dots, a_{n-1}) \cdot n^2$$

9 Programmazione dinamica

9.1 Memoizzazione

Concetto di memoizzazione per istanze ripetute

Il paradigma della programmazione dinamica può essere visto come un caso particolare di divide and conquer. Si osserva che nell'albero della ricorsione di alcuni problemi, alcune sottoistanze possono essere affrontate più volte. Il paradigma di programmazione dinamica punta ad evitare tale lavoro ridondante, attraverso la memoizzazione. Tale processo consiste nel tenere traccia delle varie sottoistanze risolte con le rispettive soluzioni, in modo da poterle riutilizzare quando si presentano nuovamente.

La presenza di sottoistanze uguali è abbastanza rara, per cui il paradigma di programmazione dinamica viene impiegato raramente e solo in determinati problemi con strutture particolari.

Approccio bottom-up iterativo e ordine di visita delle sottoistanze

In alternativa all'approccio top-down ricorsivo, è possibile risolvere i problemi di programmazione dinamica con un approccio bottom-up iterativo. Durante questo processo, si esplorano tutte le sottoistanze dal caso base fino a raggiungere l'istanza principale richiesta dall'utente e si memorizzano le soluzioni in una struttura dati, come ad esempio una tabella.

Per fare ciò, in fase di progettazione, è necessario identificare un ordine di visita delle sottoistanze in modo tale da garantire che una sottoistanza venga risolta solo dopo aver risolto tutte le sottoistanze da cui essa dipende. In altre parole è necessario individuare un ordinamento topologico delle sottoistanze.

9.2 Calcolo dell' n -esimo numero di Fibonacci

Formalizzazione del problema

L' n -esimo numero di Fibonacci è definito ricorsivamente come segue:

$$F(n) = \begin{cases} n & \text{se } n = 0, 1 \\ F(n-1) + F(n-2) & \text{se } n > 1 \end{cases} \quad \text{con} \quad \mathcal{I} = \mathbb{Z}_{+,0} \quad \mathcal{S} = \mathbb{Z}_{+,0}$$

L'istanza del problema è data da un numero n , per cui la taglia risulta essere $\log_2(n)$.

Soluzione ricorsiva non efficiente

Applicando puramente la definizione ricorsiva illustrata sopra si ottiene il seguente algoritmo.

```
0: Algoritmo: REC_FIB( $n$ )
   Input:      intero positivo  $n \in \mathbb{Z}_{+,0}$ 
   Output:     $n$ -esimo numero di Fibonacci  $F(n) \in \mathbb{Z}_+$ 

1:   if  $n = 0$  or  $n = 1$  then
2:     return  $n$ 
3:   else
4:     return REC_FIB( $n-1$ ) + REC_FIB( $n-2$ )
```

L'equazione alle ricorrenze dell'algoritmo è la seguente:

$$T(n) = \begin{cases} 0 & \text{se } n = 0, 1 \\ T(n-1) + T(n-2) + 1 & \text{se } n > 1 \end{cases}$$

Osservando che $T(n-1) \geq T(n-2)$, si calcola un lower bound della complessità temporale che risulta essere doppiamente esponenziale nella taglia dell'istanza, per cui fortemente inefficiente.

$$\begin{aligned} T(n) &\geq 2T(n-2) + 1 \\ &= \Omega\left(2^{n/2}\right) = \Omega\left(\sqrt{2}^n\right) = \Omega\left(2^{2^{(\log_2 n)-1}}\right) = \Omega\left(2^{2^{\text{taglia}(n)-1}}\right) \end{aligned}$$

Soluzione iterativa con memoizzazione

Siccome l' n -esimo numero di Fibonacci dipende solo dai due numeri precedenti, si osserva che è sufficiente tenere in memoria i due numeri di Fibonacci più recenti appena calcolati per poter calcolare il successivo. Questo tipo di approccio è anche detto bottom-up.

0: **Algoritmo:** IT_FIB(n)
 Input: intero positivo $n \in \mathbb{Z}_{+,0}$
 Output: n -esimo numero di Fibonacci $F(n) \in \mathbb{Z}_+$

1: **if** $n = 0$ **or** $n = 1$ **then**
2: **return** n
3:
4: $F[0] \leftarrow 0$
5: $F[1] \leftarrow 1$
6: **for** $i \leftarrow 2$ **to** n **to**
7: **do** $F[i] \leftarrow F[i-1] + F[i-2]$
8: **return** $F[n]$

Siccome l'algoritmo non è ricorsivo, la complessità risulta più facile da calcolare e asintoticamente è esponenziale nella taglia dell'istanza, per cui comunque poco efficiente.

$$T(n) = n + 1 = 2^{\log_2 n} - 1 = 2^{\text{taglia}(n)} - 1$$

Soluzione analitica con formula di Binet ed esponenziazione di un numero

Riscrivendo la definizione ricorsiva di Fibonacci in forma matriciale e calcolando successivamente gli autovalori e autovettori della matrice di trasferimento è possibile convertire la definizione ricorsiva in una formula chiusa nota come formula di Binet.

$$\begin{aligned} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} &= \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix} \quad \rightarrow \quad \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ \text{autovalori ed autovettori: } \lambda_1 &= \frac{1+\sqrt{5}}{2}, \quad \lambda_2 = \frac{1-\sqrt{5}}{2}, \quad v_1 = \left(\frac{1+\sqrt{5}}{2}, 1 \right), \quad v_2 = \left(\frac{1-\sqrt{5}}{2}, 1 \right) \\ \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} &= \begin{pmatrix} v_1^T \\ v_2^T \end{pmatrix} \cdot \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}^{n-1} \cdot (v_1^T \quad v_2^T)^{-1} = \\ &= \begin{pmatrix} \frac{1+\sqrt{5}}{2} & \frac{1-\sqrt{5}}{2} \\ 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} \left(\frac{1+\sqrt{5}}{2} \right)^{n-1} & 0 \\ 0 & \left(\frac{1-\sqrt{5}}{2} \right)^{n-1} \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{5}} & -\frac{1-\sqrt{5}}{2\sqrt{5}} \\ -\frac{1}{\sqrt{5}} & \frac{1+\sqrt{5}}{2\sqrt{5}} \end{pmatrix} = \\ &= \begin{pmatrix} \frac{1}{\sqrt{5}} \cdot \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right] & \dots \\ \frac{1}{\sqrt{5}} \cdot \left[\left(\frac{1+\sqrt{5}}{2} \right)^{n-1} - \left(\frac{1-\sqrt{5}}{2} \right)^{n-1} \right] & \dots \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{5}} \cdot (\lambda_1^n - \lambda_2^n) & \dots \\ \frac{1}{\sqrt{5}} \cdot (\lambda_1^{n-1} - \lambda_2^{n-1}) & \dots \end{pmatrix} \\ \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} &= \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{5}} \cdot (\lambda_1^n - \lambda_2^n) & \dots \\ \frac{1}{\sqrt{5}} \cdot (\lambda_1^{n-1} - \lambda_2^{n-1}) & \dots \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \rightarrow \quad F(n) = \frac{1}{\sqrt{5}} \cdot (\lambda_1^n - \lambda_2^n) \end{aligned}$$

La complessità della formula analitica di Binet è dominata dal calcolo di potenze di numeri reali, che risultano essere lineari nella taglia dell'istanza (per come visto in precedenza) per cui $T(n) = \Theta(\log_2 n)$.

In realtà la taglia di un numero reale non è propriamente $\log_2 n$, in quanto dipende dalla rappresentazione in virgola mobile e si necessita di un'ulteriore analisi sugli errori di rappresentazione. Per questo motivo l'algoritmo basato sulla formula di Binet si accenna solo dal punto di vista teorico.

Soluzione ricorsiva con memoizzazione

È possibile sviluppare un approccio ricorsivo con memoizzazione, in cui la funzione ricorsiva costruisce e memorizza le soluzioni intermedie in un vettore come nell'algoritmo iterativo, piuttosto che costruirle da zero ogni volta e dimenticarle. La complessità di questo approccio è uguale a quella dell'algoritmo iterativo, ovvero esponenziale nella taglia dell'istanza.

L'implementazione è composta da una funzione di inizializzazione $\text{INIT_FIB}(n)$ che costruisce il vettore dove salvare le soluzioni intermedie e da una funzione ricorsiva $\text{REC_FIB_MEM}(n)$ che costruisce le soluzioni intermedie in modo ricorsivo memorizzandole nel vettore.

0: **Algoritmo:** $\text{INIT_FIB}(n)$

Input: intero positivo $n \in \mathbb{Z}_{+,0}$

Output: n -esimo numero di Fibonacci $F(n) \in \mathbb{Z}_+$

```
1:  if  $n = 0$  or  $n = 1$  then
2:      return  $n$ 
3:
4:       $F[0] \leftarrow 0$ 
5:       $F[1] \leftarrow 1$ 
6:      for  $i \leftarrow 2$  to  $n$  to
7:          do  $F[i] \leftarrow 0$ 
8:      return  $\text{REC\_FIB\_MEM}n$ 
```

0: **Algoritmo:** $\text{REC_FIB_MEM}(n)$

Input: intero positivo $n \in \mathbb{Z}_{+,0}$

Output: n -esimo numero di Fibonacci $F(n) \in \mathbb{Z}_+$

```
1:  if  $F[n] = 0$  then
2:       $F[n] \leftarrow \text{REC\_FIB\_MEM}(n-1) + \text{REC\_FIB\_MEM}(n-2)$ 
3:      return  $F[n]$ 
```

9.3 Longest Common Subsequence - LCS

Definizione di sottosequenza e di sottostringa di una stringa

La sottosequenza di una stringa X è una sequenza di simboli non necessariamente contigui, ordinati con lo stesso ordine con cui sono presenti nella stringa X . Una sottostringa è una sottosequenza in cui è richiesto che i simboli devono essere contigui, mentre una sottosequenza non richiede tale contiguità.

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad \# \text{ di sottosequenze di lunghezza } k \text{ di una stringa di lunghezza } n$$

$$\sum_{k=0}^n \binom{n}{k} = 2^n \quad \# \text{ di tutte le sottosequenze di una stringa di lunghezza } n$$

Si nota che per ottenere una sottosequenza di lunghezza k , si scelgono a caso k simboli dalla stringa di lunghezza n senza tenere conto dell'ordine. È equivalente a cercare un sottoinsieme di k elementi da un insieme di n elementi, per questo motivo si utilizza il coefficiente binomiale. L'ordine dei simboli nella sottosequenza è imposto a posteriori in base all'ordine con cui sono presenti nella stringa originale.

Formalizzazione del problema

Date due stringhe X e Y rispettivamente di lunghezza m e n , si vuole calcolare la sottosequenza più lunga Z che sia comune ad entrambe le stringhe X e Y . La taglia dell'istanza è data da $m + n$.

Descrizione dell'algoritmo $\text{LCS}(X, Y)$ ricorsivo con memoizzazione

- **caso base:**
se una delle due stringhe X o Y è vuota, la sottosequenza comune più lunga è anch'essa vuota
- **passo ricorsivo - caso 1:**
se le due stringhe $X = \langle X', a \rangle$ e $Y = \langle Y', a \rangle$ terminano con lo stesso simbolo a , allora la sottosequenza comune più lunga è data dalla sottosequenza comune più lunga tra X' e Y' ($\text{LCS}(X', Y')$) con l'aggiunta del simbolo a alla fine
- **passo ricorsivo - caso 2:**
se le due stringhe $X = \langle X', a \rangle$ e $Y = \langle Y', b \rangle$ terminano con simboli diversi a e b , allora la sottosequenza comune più lunga sarà data dalla massima tra la sottosequenza comune più lunga tra X' e Y ($\text{LCS}(X', Y)$) e quella tra X e Y' ($\text{LCS}(X, Y')$)

Per identificare i prefissi di X e Y da passare alle chiamate ricorsive, si utilizzano rispettivamente gli indici i e j che puntano all'ultimo simbolo di X e Y da considerare. Di seguito sono riportate la proprietà di sottostruttura e il calcolo della lunghezza l della LCS indicate come equazioni alle ricorrenze.

$$Z(X_{0:i}, Y_{0:j}) = \text{LCS}(i, j) = \begin{cases} \varepsilon & \text{caso base - per } i = 0 \vee j = 0 \\ \text{LCS}(i-1, j-1) \cup \langle X_i \rangle & \text{caso 1 - se } X_i = Y_j \\ \max\{\text{LCS}(i-1, j), \text{LCS}(i, j-1)\} & \text{caso 2 - se } X_i \neq Y_j \end{cases}$$

$$\text{len}(Z(X_{0:i}, Y_{0:j})) = l(i, j) = \begin{cases} 0 & \text{caso base - per } i = 0 \vee j = 0 \\ l(i-1, j-1) + 1 & \text{caso 1 - se } X_i = Y_j \\ \max\{l(i-1, j), l(i, j-1)\} & \text{caso 2 - se } X_i \neq Y_j \end{cases}$$

L'algoritmo viene definito con approccio bottom-up, ovvero si parte da un caso base semplice $i = j = 0$ e si espandono le soluzioni intermedie facendo variare progressivamente un indice alla volta fino ad arrivare alla soluzione finale $i = m$ e $j = n$. Ogni soluzione intermedia viene memorizzata in una tabella bidimensionale, come illustrato successivamente.

Tabella dei risultati (struttura dati di memoizzazione)

Le soluzioni intermedie vengono memorizzate in una tabella bidimensionale di dimensione $(m+1) \times (n+1)$ con le seguenti caratteristiche:

- la cella (i, j) fa riferimento alla chiamata $LCS(i, j)$
- il valore memorizzato nella cella (i, j) è la lunghezza $l(i, j)$ della $LCS(i, j)$
- la soluzione finale $LCS(X, Y)$ è contenuta nella cella (m, n) (in basso a destra) della tabella
- in ogni cella è riportata una freccia che indica la cella riferita alla chiamata ricorsiva effettuata per calcolare il valore della cella stessa, con le seguenti regole:
 - se $X_i = Y_j$ la freccia è diagonale e punta alla cella $i - 1, j - 1$
 - se $X_i \neq Y_j$ la freccia è orizzontale o verticale e punta rispettivamente alla cella $i - 1, j$ o alla cella $i, j - 1$ a seconda di quale delle due chiamate ricorsive ha restituito il valore più grande (alcune volte è indipendente scegliere tra orizzontale o verticale se i valori sono uguali)
- per l'approccio di costruzione bottom-up, il popolamento della tabella parte dal caso base $(0, 0)$ (in alto a sinistra) e prosegue per righe (approccio row-major) oppure per colonne (approccio column-major), in base a che indice si decide di variare per primo

Di seguito ne è riportato un esempio per le stringhe $X = \langle A, B, C, B, B, D \rangle$ e $Y = \langle A, D, C, C, B, D \rangle$.

$\begin{matrix} \backslash & y \\ x & \end{matrix}$	Y_0 ε	Y_1 A	Y_2 D	Y_3 C	Y_4 C	Y_5 B	Y_6 D
$X_0 : \varepsilon$	0	0	0	0	0	0	0
$X_1 : A$	0	$\swarrow 1$	$\leftarrow 1$	$\leftarrow 1$	$\leftarrow 1$	1	1
$X_2 : B$	0	1	1	1	$\uparrow 1$	2	2
$X_3 : C$	0	1	1	2	$\swarrow 2$	2	2
$X_4 : B$	0	1	1	2	$\uparrow 2$	3	3
$X_5 : B$	0	1	1	2	2	$\swarrow 3$	3
$X_6 : D$	0	1	2	2	2	3	$\swarrow 4$

Ricostruzione della LCS a partire dalle frecce della tabella

La ricostruzione della sottosequenza comune più lunga Z equivale ad una visita in post-order dell'ordinamento topologico delle frecce contenute nella tabella b . La prima chiamata deve essere effettuata con gli indici $i = m$ e $j = n$ che puntano all'ultima cella della tabella. La visita termina quando si raggiunge una cella con indice $i = 0$ o $j = 0$ (caso base).

0: **Algoritmo:** PRINT_LCS($\underline{X}, \underline{b}, i, j$)

Input: stringa $\underline{X}$ per recupero elementi, tabella $\underline{b}$ con le frecce e indici i, j

Output: nessun return, stampa la LCS(X, Y) a standard output

-
- ```

1: if $m = 0$ or $n = 0$ then $\triangleright$ invocazione sul caso base $\rightarrow$ nessuna azione necessaria
2: return -
3: if $b[i, j] = \swarrow$ then $\triangleright$ caso 1 - cella diagonale
4: PRINT_LCS($\underline{X}, \underline{b}, i - 1, j - 1$)
5: print(X_i)
6: else if $b[i, j] = \uparrow$ then $\triangleright$ caso 2 - cella verticale
7: PRINT_LCS($\underline{X}, \underline{b}, i - 1, j$)
8: else (ovvero per $b[i, j] = \leftarrow$) $\triangleright$ caso 2 - cella orizzontale
9: PRINT_LCS($\underline{X}, \underline{b}, i, j - 1$)

```
-

## Implementazione iterativa bottom-up

0: **Algoritmo:** IT\_LCS( $\underline{X}, \underline{Y}$ )

**Input:** stringhe  $\underline{X}$  e  $\underline{Y}$  rispettivamente di lunghezza  $m$  e  $n$

**Output:**  $(l(m, n), \underline{b})$  lunghezza e tabella con le frecce per ricostruire la LCS( $X, Y$ )

```
1: $m \leftarrow \text{len}(X)$
2: $n \leftarrow \text{len}(Y)$
3: for $i \leftarrow 0$ to m do $l[i, 0] \leftarrow 0$ ▷ inizializzazione caso base della prima riga
4: for $j \leftarrow 0$ to n do $l[0, j] \leftarrow 0$ ▷ inizializzazione caso base della prima colonna
5:
6: for $i \leftarrow 1$ to m do ▷ popolamento della tabella row-major
7: for $j \leftarrow 1$ to n do
8: if $X_i = Y_j$ then ▷ passo ricorsivo - caso 1
9: $l[i, j] \leftarrow l[i - 1, j - 1] + 1$ $\quad b[i, j] \leftarrow \nwarrow$
10: else if $l[i - 1, j] \geq l[i, j - 1]$ then ▷ passo ricorsivo - caso 2 - cella verticale
11: $l[i, j] \leftarrow l[i - 1, j]$ $\quad b[i, j] \leftarrow \uparrow$
12: else if $l[i - 1, j] < l[i, j - 1]$ then ▷ passo ricorsivo - caso 2 - cella orizzontale
13: $l[i, j] \leftarrow l[i, j - 1]$ $\quad b[i, j] \leftarrow \leftarrow$
14: $b[i, j] \leftarrow \leftarrow$
15: return $(l[m][n], \underline{b})$
```

## Implementazione ricorsiva top-down con memoizzazione

0: **Algoritmo:** INIT\_LCS( $\underline{X}, \underline{Y}$ )

**Input:** stringhe  $\underline{X}$  e  $\underline{Y}$  rispettivamente di lunghezza  $m$  e  $n$

**Output:**  $(l(m, n), \underline{b})$  lunghezza e tabella con le frecce per ricostruire la LCS( $X, Y$ )

```
1: $m \leftarrow \text{len}(X)$
2: $n \leftarrow \text{len}(Y)$
3: if $m = 0$ or $n = 0$ then ▷ invocazione sul caso base $\rightarrow$ nessuna azione necessaria
4: return 0
5: for $i \leftarrow 0$ to m do $l[i, 0] \leftarrow 0$ ▷ inizializzazione caso base della prima riga
6: for $j \leftarrow 0$ to n do $l[0, j] \leftarrow 0$ ▷ inizializzazione caso base della prima colonna
7: for $i \leftarrow 1$ to m do ▷ inizializzazione con valore sentinella -1
8: for $j \leftarrow 1$ to n do ▷ per identificare i casi non ancora gestiti
9: $l[i, j] \leftarrow -1$
10: return REC_LCS($\underline{X}, \underline{Y}, m, n$)
```

0: **Algoritmo:** REC\_LCS( $\underline{X}, \underline{Y}, m, n$ )

**Input:** stringhe  $\underline{X}$  e  $\underline{Y}$  e rispettive lunghezze  $m$  e  $n$

**Output:**  $(l(m, n), \underline{b})$  lunghezza e tabella con le frecce per ricostruire la LCS( $X, Y$ )

```
1: if $l[m, n] = -1$ then ▷ caso non ancora gestito
2: if $X_m = Y_n$ then ▷ passo ricorsivo - caso 1
3: $l[m, n] \leftarrow 1 + \text{REC_LCS}(\underline{X}, \underline{Y}, m - 1, n - 1)$ $\quad b[m, n] \leftarrow \nwarrow$
4: else if $\text{REC_LCS}(\underline{X}, \underline{Y}, m - 1, n) \geq \text{REC_LCS}(\underline{X}, \underline{Y}, m, n - 1)$ then ▷ caso 2 - cella verticale
5: $l[m, n] \leftarrow l[m - 1, n]$ $\quad b[m, n] \leftarrow \uparrow$
6: else (ovvero per $l[m - 1, n] < l[m, n - 1]$) ▷ caso 2 - cella orizzontale
7: $l[m, n] \leftarrow l[m, n - 1]$ $\quad b[m, n] \leftarrow \leftarrow$
8: return $(l[m][n], \underline{b})$
```

## Complessità temporale

La complessità temporale dell'algoritmo LCS è  $O(m \cdot n)$  sia per l'implementazione iterativa che per quella ricorsiva con memoizzazione, per come di seguito spiegato:

- nella versione iterativa, si visitano tutte le  $n \cdot m$  celle della tabella e per ogni cella viene fatto un numero costante di operazioni
- nella versione ricorsiva con memoizzazione, ogni cella della tabella deve venire inizializzata per cui il ragionamento è lo stesso della versione iterativa

Tale complessità, seppur quadratica per  $m = n$ , è comunque molto più efficiente rispetto alla complessità esponenziale dell'algoritmo banale brute-force che esplora tutte le possibili sottosequenze di  $X$  e  $Y$  per identificare quella in comune più lunga.

Con alcuni accorgimenti è possibile ridurre la complessità temporale fino a  $O(m \cdot n / \log_2 n)$ .

## NO-SETH e Fine Grained Complexity

Il problema LCS è definito come NO-SETH, ovvero è un algoritmo che se dovesse essere risolto in tempo polinomiale subquadratico  $O(n^{2-\epsilon})$ , allora sarebbe possibile risolvere in tempo polinomiale molti altri problemi che attualmente hanno solo soluzioni con complessità esponenziale. Questo studio è svolto nell'ambito della "Fine Grained Complexity", ovvero una branca dello studio della complessità che si occupa di identificare i valori precisi delle complessità di algoritmi per studiare come piccoli miglioramenti su alcuni algoritmi possano portare al passaggio da esponenziale a polinomiale per molti altri problemi.

## Complessità spaziale

La complessità spaziale dell'algoritmo LCS è  $O(m \cdot n)$ , ovvero è proporzionale al numero di celle della tabella in cui vengono salvati i risultati. Corrisponde a tutte le possibili combinazioni di prefissi di  $X$  e  $Y$  che possono essere passati alle chiamate ricorsive.

Se si è interessati a conoscere solo la lunghezza della LCS, è possibile ridurre la complessità spaziale a  $O(2 \cdot \min\{m, n\})$  memorizzando soltanto le due righe (o colonne) più recenti della tabella (quella precedente e quella che si sta popolando), in quanto le equazioni alle ricorrenze dipendono solo dai valori delle celle adiacenti (diagonale, orizzontale e verticale).

## Osservazioni sulla struttura dati per la memoizzazione

È richiesta una struttura dati con le seguenti operazioni:

- ricerca tramite la sottoistanza (coppia di indici  $(i, j)$ ) che restituisce  $-1$  se la sottoistanza cercata non è ancora stata gestita
- inserimento delle soluzioni per le sottoistanze risolte

La struttura più semplice che garantisce tali operazioni con complessità  $O(1)$  è una matrice bidimensionale, ma ha lo svantaggio che lo spazio occupato è fisso di  $O(m \cdot n)$  indipendentemente dal numero di sottoistanze effettivamente gestite.

Un'alternativa più efficiente a livello spaziale sono il dizionario o mappa (eventualmente semplificati in quanto non è richiesta la cancellazione) in quanto occupano lo spazio necessario per memorizzare solo le sottoistanze effettivamente gestite. Lo svantaggio è che hanno complessità di ricerca e inserimento di  $O(\log_2(m \cdot n))$ . Risultano, quindi, più efficienti solo se il numero di sottoistanze effettivamente gestite è minore di  $(m \cdot n) / \log_2(m \cdot n)$ .

## 9.4 Matrix Chain Multiplication - MCM

### Introduzione al problema

Dati tre vettori  $p$ ,  $q$  e  $r$  rispettivamente di dimensione  $n \times 1$  (colonna),  $1 \times n$  (riga) e  $n \times 1$  (colonna), si vuole trovare una parentesizzazione del loro prodotto per risolverlo in maniera efficiente.

$$(p \cdot q) \cdot r = (\text{colonna} \cdot \text{riga}) \cdot \text{colonna} = \text{matrice} \cdot \text{colonna} \rightarrow T(n) = \underbrace{n^2}_{p_{\text{colonna}} \cdot q_{\text{riga}}} + \underbrace{2n-1}_{\text{matrice} \cdot r_{\text{colonna}}}$$

$$p \cdot (q \cdot r) = \text{colonna} \cdot (\text{riga} \cdot \text{colonna}) = \text{colonna} \cdot \text{scalare} \rightarrow T(n) = \underbrace{n}_{p_{\text{colonna}} \cdot \text{scalare}} + \underbrace{2n-1}_{q_{\text{riga}} \cdot r_{\text{colonna}}}$$

Si osserva che parentesizzando in modo diverso, si ottengono complessità decisamente diverse. Elencare tutte le possibili parentesizzazioni richiederebbe costi esponenziali, per cui è necessario identificare un approccio più efficiente.

### Formalizzazione del problema

L'input del problema è una sequenza di  $n+1$  interi  $p_0, p_1, \dots, p_{n-1}, p_n$  che rappresentano le dimensioni delle  $n$  matrici da moltiplicare, secondo la relazione  $A_i \in \mathbb{C}^{p_{i-1} \times p_i}$  con  $i = 1, \dots, n$ .

Inoltre si rappresenta una parentesizzazione di una sequenza di matrici  $A_i, A_{i+1}, \dots, A_j$  attraverso la coppia di indici  $i$  e  $j$  che identificano la prima e l'ultima matrice della sottostringa.

### Idea risolutiva D&C e presenza di istanze sovrapposte

Data una stringa di  $j-i+1$  matrici  $\langle A_i, A_{i+1}, \dots, A_j \rangle$  delimitate dagli indici  $i$  e  $j$ , si identifica la posizione  $k$  ottimale in cui dividere o parentesizzare tale stringa generando due sottostringhe  $\langle A_i, A_{i+1}, \dots, A_k \rangle$  e  $\langle A_{k+1}, A_{k+2}, \dots, A_j \rangle$  delimitate rispettivamente dalle coppie di indici  $(i, k)$  e  $(k+1, j)$ . Una volta fatto ciò, si risolve ricorsivamente il problema per le due sottostringhe ottenendo due matrici  $A_{i:k}$  e  $A_{k+1:j}$  che poi verranno moltiplicate tra loro per ottenere la matrice  $A_{i:j}$  corrispondente al prodotto di tutte le matrici della stringa originale.

Si definisce la funzione  $c[i, j]$  come il costo minimo del prodotto delle matrici della sottostringa  $\langle A_i, A_{i+1}, \dots, A_j \rangle$  supponendo di aver scelto il valore  $k$  ottimo per minimizzare tale costo. È utile notare che il costo del prodotto tra le due matrici  $A_{i:k} \in \mathbb{C}^{p_{i-1} \times p_k}$  e  $A_{k+1:j} \in \mathbb{C}^{p_k \times p_j}$  è dato da  $p_{i-1} \cdot p_k \cdot p_j$ .

$$c[i, j] = \begin{cases} 0 & \text{se } i = j \\ \min_{i \leq k < j} \{ c[i, k] + c[k+1, j] + p_{i-1} \cdot p_k \cdot p_j \} & \text{se } i < j \end{cases}$$

Si osserva, però, che alcune sottostringhe di matrici possono comparire più volte nell'albero delle chiamate ricorsive, per cui un approccio puramente ricorsivo con D&C risulterebbe inefficiente. Si preferisce quindi sfruttare il paradigma della programmazione dinamica.

### Soluzione tramite memoizzazione

Come per il problema LCS, si memorizzano le soluzioni delle sottoistanze identificate dalle coppie di indici  $(i, j)$  all'interno delle celle di una tabella bidimensionale  $m$  di dimensione  $n \times n$ . Si osserva che la tabella ha le seguenti caratteristiche:

- siccome  $i \leq j$ , le uniche celle della tabella che vengono effettivamente utilizzate sono quelle sopra la diagonale principale, per cui la tabella è triangolare superiore
- le celle lungo la diagonale principale  $c[i, j]$  con  $i = j$  contengono il costo 0, ovvero il caso base
- le celle appena sopra alla diagonale principale  $c[i, j]$  con  $j - i = 1$  riflettono tutte le sottostringhe di due matrici, le celle appena sopra a queste riflettono tutte le sottostringhe di tre matrici e così via
- la cella  $c[1, n]$  in alto a destra contiene la soluzione finale, ovvero il costo minimo del prodotto di tutte le matrici della stringa in input

## Implementazione iterativa

---

0: **Algoritmo:** IT\_MCM( $\underline{p}$ )

**Input:** vettore  $\underline{p}$  di dimensione  $n + 1$  con le dimensioni delle matrici

**Output:**  $m[1, n]$  costo minimo come definito da  $c[1, n]$ , matrice  $\underline{s}$  con i valori di  $k$  ottimali

```
1: $n \leftarrow \text{len}(\underline{p}) - 1$
2: for $i \leftarrow 1$ to n do $m[i, i] \leftarrow 0$ ▷ inizializzazione caso base della diagonale principale
3: for $l \leftarrow 2$ to n do ▷ per ogni lunghezza l della sottostringa di matrici
4: for $i \leftarrow 1$ to $n - l + 1$ do ▷ discesa lungo la diagonale delle sottostringhe di lunghezza l
5: $j \leftarrow i + l - 1$
6: $m[i, j] \leftarrow +\infty$ ▷ inizializzazione con valore sentinella
7: for $k \leftarrow i$ to $j - 1$ do ▷ per ogni possibile punto di divisione k
8: $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1} \cdot p_k \cdot p_j$ ▷ calcolo del costo per la divisione in k
9: if $q < m[i, j]$ then ▷ se il costo calcolato è minore del costo attuale memorizzato
10: $m[i, j] \leftarrow q$ ▷ aggiornamento del costo minimo per la sottostringa i, j
11: $s[i, j] \leftarrow k$ ▷ memorizzazione del punto di divisione ottimo per la sottostringa i, j
12: return ($m[1, n], \underline{s}$)
```

---

## Alcune osservazioni sull'implementazione iterativa

- il popolamento della tabella avviene per diagonali, partendo da quella principale e procedendo verso l'alto a destra: il primo “for” varia la lunghezza  $l$  (sceglie la diagonale), il secondo “for” varia l'indice  $i$  (sceglie la riga della cella da popolare) e l'indice  $j$  viene calcolato di conseguenza in base a  $i$  e  $l$
- in presenza di parentesizzazioni equivalenti (ad esempio per matrici quadrate), l'algoritmo individua quella con valore di  $k$  minimo, in quanto utilizzando l'operatore  $<$  e non  $<=$  nell'if a riga 9, le soluzioni equivalenti non vengono considerate come ottimali
- al posto di avere due tabelle triangolari superiori  $m$  e  $s$ , è possibile utilizzare la stessa tabella  $m$  popolata superiormente con i costi e inferiormente con i valori di  $k$  ottimali

## Complessità spaziale e temporale

La complessità temporale dell'algoritmo MCM è  $O(n^3)$  in quanto si visitano circa la metà delle celle della tabella che asintoticamente sono  $O(n^2)$  e per ciascuna di esse si esegue un ciclo su  $k$  che varia da  $i$  a  $j$  con  $i, j \in [1, n]$  in cui si svolgono operazioni in tempo costante.

La complessità spaziale dell'algoritmo MCM è  $O(n^2)$  in quanto le tabelle impiegate devono avere dimensione  $n \times n$  per memorizzare i costi e le posizioni di parentesizzazione ottimali per tutte le possibili coppie di indici  $i$  e  $j$ . A differenza dell'algoritmo di LCS, il valore di una cella  $m[i, j]$  non dipende esclusivamente dalle celle adiacenti, per cui non si possono fare ottimizzazioni a livello spaziale.

## 9.5 Longest Palindrome Subsequence - LPS

### Definizione di sequenza palindroma

Una sequenza  $Z = \langle z_1, z_2, \dots, z_n \rangle$  si dice palindroma se  $z_{1+h} = z_{n-h}$  per ogni  $h = 0, 1, \dots, n-1$ . Si ricorda che una sottosequenza non necessariamente deve contenere simboli contigui.

### Definizione della proprietà di sottostruttura

Per definire la più lunga sottosequenza palindroma di una sequenza  $Z_{i:j}$ , si sceglie di confrontare i due simboli agli estremi  $z_i$  e  $z_j$ :

- se sono uguali, allora la sottosequenza palindroma conterrà tali due simboli oltre alla più lunga sottosequenza palindroma della sottostringa  $Z_{i+1:j-1}$
- se sono diversi, allora la sottosequenza palindroma non conterrà almeno uno dei due simboli, per cui si sceglierà la più lunga tra quella della sottostringa  $Z_{i+1:j}$  e quella della sottostringa  $Z_{i:j-1}$

Di seguito si formalizza la proprietà di sottostruttura in termini di equazioni alle ricorrenze.

$$LPS(Z_{i:j}) = \begin{cases} \{z_i\} & \text{se } i = j \\ \{z_i, z_j\} & \text{se } j - i = 1 \text{ e } z_i = z_j \\ \{z_i\} \text{ oppure } \{z_j\} & \text{se } j - i = 1 \text{ e } z_i \neq z_j \\ \{z_i\} \cup LPS(Z_{i+1:j-1}) \cup \{z_j\} & \text{se } j - i > 1 \text{ e } z_i = z_j \\ \argmax_{\substack{LPS(Z_{i+1:j}) \\ LPS(Z_{i:j-1})}} \{ \text{len}(LPS(Z_{i+1:j})), \text{len}(LPS(Z_{i:j-1})) \} & \text{se } j - i > 1 \text{ e } z_i \neq z_j \end{cases}$$

In alternativa, si può definire la proprietà di sottostruttura in termini di lunghezza della sottosequenza palindroma, nel caso in cui si è interessati solo alla sua lunghezza e non ai simboli che la compongono.

$$p(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } j - i = 1 \text{ e } z_i = z_j \\ 1 & \text{se } j - i = 1 \text{ e } z_i \neq z_j \\ p(i+1, j-1) + 2 & \text{se } j - i > 1 \text{ e } z_i = z_j \\ \max \{p(i+1, j), p(i, j-1)\} & \text{se } j - i > 1 \text{ e } z_i \neq z_j \end{cases}$$

### Note sull'implementazione

L'implementazione iterativa bottom-up a pagina seguente è molto simile a quella dell'algoritmo per la Longest Common Subsequence (LCS) tra due stringhe. Come per l'LCS, si utilizza una tabella  $p$  per memorizzare la lunghezza della sottosequenza palindroma più lunga per ogni sottostringa  $Z_{i:j}$  e una tabella  $b$  per memorizzare che cella adiacente è associata alla sottostringa generata. La principale differenza è che la tabella è riempita a diagonali come nella Matrix Chain Multiplication (MCM).

Per ricostruire la sottosequenza palindroma più lunga, si parte dalla cella  $p[1, n]$  che contiene la lunghezza della sottosequenza palindroma più lunga per l'intera stringa  $Z$  e si procede a scendere nella tabella  $p$  seguendo le direzioni memorizzate nella tabella  $b$  fino ad arrivare al caso base nelle celle della diagonale  $p[i, i]$ .

### Complessità computazionale e spaziale

La complessità computazionale è  $T(n) = \Theta(n^2)$  perché si devono visitare tutte le celle della tabella  $p$  di dimensione  $n \times n$  e per ogni cella si effettuano un numero costante di operazioni.

La complessità spaziale è  $S(n) = \Theta(n^2)$  per la tabella  $p$  di dimensione  $n \times n$  e per la tabella  $b$  di dimensioni analoghe. Come per la LCS, se si è interessati solo alla lunghezza della sottosequenza palindroma più lunga è possibile ridurre la complessità spaziale a  $S(n) = \Theta(n)$  memorizzando solo le due diagonali della tabella  $p$  necessarie per il calcolo iterativo.



## Implementazione iterativa

---

0: **Algoritmo:** IT\_LPS( $\underline{x}$ )

**Input:** sequenza  $\underline{x}$

**Output:** lunghezza della LPS di  $\underline{x}$  e tabella delle transizioni  $\underline{b}$  per ricostruire la LPS

```
1: $n \leftarrow \text{len}(\underline{x})$
2: for $i \leftarrow 1$ to $n - 1$ do ▷ inizializzazione dei casi base per le sottosequenze di lunghezza 1 e 2
3: $p[i, i] \leftarrow 1$
4: if $x[i] = x[i + 1]$ then $p[i, i + 1] \leftarrow 2$
5: else $p[i, i + 1] \leftarrow 1$
6: $p[n, n] \leftarrow 1$
7:
8: for $l \leftarrow 3$ to n do ▷ calcolo iterativo delle sottosequenze di lunghezza crescente
9: for $i \leftarrow 1$ to $n - l + 1$ do
10: $j \leftarrow i + l - 1$
11: if $x[i] = x[j]$ then
12: $p[i, j] \leftarrow p[i + 1, j - 1] + 2$
13: $b[i, j] \leftarrow \text{"\text{↖"}}$
14: else if $p[i + 1, j] \geq p[i, j - 1]$ then
15: $p[i, j] \leftarrow p[i + 1, j]$
16: $b[i, j] \leftarrow \text{"\text{↓"}}$
17: else (ovvero per $p[i + 1, j] < p[i, j - 1]$)
18: $p[i, j] \leftarrow p[i, j - 1]$
19: $b[i, j] \leftarrow \text{"\text{←"}}$
20: return $p[1, n]$, $\underline{b}$
```

---

0: **Algoritmo:** PRINT\_LPS( $\underline{x}$ ,  $\underline{b}$ ,  $i$ ,  $j$ )

**Input:** sequenza  $\underline{x}$ , tabella delle transizioni  $\underline{b}$ , indici  $i$  e  $j$

**Output:** nessun return, stampa la LPS( $\underline{x}$ ) a standard output

```
1: if $i = j$ then print($x[i]$) ▷ caso base: sottosequenza di lunghezza 1
2: if $j - i = 1$ then ▷ caso base: sottosequenza di lunghezza 2
3: if $x[i] = x[j]$ then print($x[i], x[j]$)
4: else print($x[i]$)
5:
6: if $b[i, j] = \text{"\text{↖"}}$ then ▷ simboli agli estremi uguali
7: print($x[i]$)
8: PRINT_LPS($\underline{x}$, $\underline{b}$, $i + 1$, $j - 1$)
9: print($x[j]$)
10: else if $b[i, j] = \text{"\text{↓"}}$ then ▷ caso 1 - cella sotto
11: PRINT_LPS($\underline{x}$, $\underline{b}$, $i + 1$, j)
12: else (ovvero per $b[i, j] = \text{"\text{←"}}$) ▷ caso 2 - cella a sinistra
13: PRINT_LPS($\underline{x}$, $\underline{b}$, i , $j - 1$)
```

---

## 10 Paradigma Greedy

### 10.1 Proprietà della scelta greedy

#### Scelta greedy

Il paradigma greedy è un caso particolare di divide and conquer, in cui la soluzione di un'istanza viene costruita ad ogni passo induttivo, attraverso la scelta di un elemento ottimo locale, con l'obiettivo di ottenere una soluzione globale ottima. La scelta effettuata è detta scelta greedy e spesso è estremamente banale (con costo costante), la parte complessa è dimostrare che la scelta greedy porta ad una soluzione globale ottima.

#### Proprietà di scelta greedy

La proprietà di scelta greedy è la condizione che deve soddisfare una scelta greedy per garantire che la scelta effettuata localmente non comprometta l'ottimalità della soluzione globale. Una scelta greedy riduce sia l'istanza da risolvere che lo spazio generale delle soluzioni (ottimali e subottimali). Per portare alla soluzione (e soddisfare la proprietà di scelta greedy) è necessario che lo spazio delle soluzioni, dopo la riduzione, contenga ancora almeno una soluzione ottima globale.

#### Dimostrazione della correttezza di un algoritmo greedy

A livello pratico, per dimostrare che una scelta greedy soddisfa la proprietà di scelta greedy, è necessario dimostrare che esiste una soluzione ottima globale che soddisfi la scelta greedy. Spesso si considera una generica soluzione ottima globale e, se tale soluzione non contiene la scelta greedy, si dimostra che è possibile modificarla senza comprometterne l'ottimalità, in modo che soddisfi la scelta greedy.

#### Algoritmi greedy subottimali

Esistono alcuni problemi per cui sono stati sviluppati algoritmi greedy subottimali, che non garantiscono in assoluto il raggiungimento della soluzione ottima globale, ma la raggiungono o ci si avvicinano molto con alta probabilità. Questi algoritmi risultano essere molto più veloci degli algoritmi ottimi e sono impiegati in cui si è disposti a sacrificare un po' di ottimalità in cambio di una maggiore velocità di esecuzione.

### 10.2 Scheduling di attività

#### Formalizzazione del problema

Data una sequenza  $S = \{a_1, a_2, \dots, a_n\}$  di  $n$  attività  $a_i : [s_i, f_i)$ , identificate dal proprio istante di inizio  $s_i$  e di fine  $f_i$  con  $s_i < f_i$ , si vuole individuare un sottoinsieme fattibile  $A \subseteq S$  di dimensione massima.

- un sottoinsieme  $A \subseteq S$  di attività è fattibile se composto da attività mutualmente compatibili
- due attività  $a_i$  e  $a_j$  sono compatibili se e solo se i loro intervalli di tempo non si sovrappongono:

$$a_i : [s_i, f_i) \cap a_j : [s_j, f_j) = \emptyset \iff s_i \geq f_j \vee s_j \geq f_i$$

#### Definizione della scelta greedy

Data una sequenza  $S$  di  $n$  attività, si costruisce l'insieme soluzione  $A$  scegliendo l'attività  $a_i \in S$  che termina per prima (con  $f_i = \min_j f_j$ ) unita all'insieme soluzione  $A_{\text{sub}}$  della sottoistanza  $S_{\text{sub}}$  ottenuta partendo da  $S$  e rimuovendo tutte le attività incompatibili con l'attività scelta  $a_i$ .

Di seguito si formalizza la scelta greedy attraverso la proprietà di sottostruttura espressa in termini di equazioni alle ricorrenze.

$$A(S) = \begin{cases} \{a_1\} & \text{se } |S| = n = 1 \\ \{\{a_k \in S \text{ con } k = \operatorname{argmin} f_i\} \cup A(S - \{a_l \in S \text{ incompatibili con } a_k\})\} & \text{se } |S| = n > 1 \end{cases}$$

### Algoritmo ricorsivo ottimizzato basato sulla scelta greedy

Per rendere più pratica l'operazione di scelta dell'attività ottima, si ordina la sequenza  $S$  in ordine crescente rispetto agli istanti di fine  $f_i$  delle varie attività, con costo di ordinamento  $\Theta(n \log_2 n)$ . In questo modo l'attività da scegliere sarà sempre la prima della sequenza  $S$  ordinata.

Inoltre si osserva che non è necessario visitare tutta la sequenza  $S$  per rimuovere le tutte le attività incompatibili ogni volta che viene scelta un'attività  $a_i$ . È sufficiente fermarsi non appena si trova la prima attività compatibile  $a_j$  che sarà l'attività successiva da aggiungere all'insieme soluzione  $A$ . Le attività incompatibili con  $a_i$  saranno anche incompatibili con  $a_j$ , per cui verranno rimosse successivamente senza influenzare la scelta ottima.

---

0: **Algoritmo:** REC\_GREEDY\_SEL( $\underline{s}, \underline{f}, i$ )

**Input:**  $\underline{s}$  e  $\underline{f}$  istanti di inizio e fine ordinati per  $f_i$ , indice  $i$  della prima attività compatibile

**Output:**  $A = \{a_{i_1}, a_{i_2}, \dots, a_{i_k}\} \subseteq S$  fattibile e di dimensione massima

```
1: $n \leftarrow \text{len}(\underline{s})$
2: if $i > n$ then return $\emptyset$ $\triangleright$ caso base: non ci sono più attività da considerare
3: $A \leftarrow \{i\}$ $\triangleright$ scelta greedy: si sceglie la prima attività della sequenza
4: $j \leftarrow i + 1$
5: while $j \leq n$ and $\underline{s}[j] < \underline{f}[i]$ do $j \leftarrow j + 1$ $\triangleright$ si saltano le attività incompatibili con a_i
6: return $A \cup \text{REC_GREEDY_SEL}(\underline{s}, \underline{f}, j)$ $\triangleright$ proprietà di sottostruttura con chiamata ricorsiva
```

---

### Dimostrazione della correttezza - verifica della proprietà greedy

Di seguito si dimostra che la scelta greedy ideata rispetta la proprietà greedy, ovvero che esiste una soluzione ottima  $A^*$  che rispetta la scelta greedy dell'algoritmo.

Si suppone di avere una generica soluzione ottima  $A^*$ :

- se la prima attività  $a_1$  della soluzione ottima  $A^*$  è  $a_k$  con  $k = \text{argmin}_i f_i$ , allora la scelta greedy è banalmente rispettata
- se la prima attività  $a_1$  della soluzione ottima  $A^*$  non è  $a_k$  con  $k = \text{argmin}_i f_i$ , vuol dire che  $f_1 > f_k$ , per cui è possibile sostituire  $a_1$  con  $a_k$  senza perdere fattibilità, ottenendo una nuova soluzione che rispetta la scelta greedy e rimane soluzione ottima

### Analisi della complessità

Escludendo il costo dell'ordinamento iniziale  $\Theta(n \log_2 n)$ , la complessità dell'algoritmo è data soltanto dalle operazioni di confronto durante la visita della sequenza  $S$  per saltare le attività incompatibili. Siccome le attività della sequenza vengono visitate in maniera sequenziale e ad ogni visita vengono effettuate un numero costante di operazioni, la complessità asintotica dell'algoritmo è  $T(n) = \Theta(n)$ .

## 10.3 Codice di Huffman

### Introduzione al codice di Huffman

Il codice di Huffman è un codice a prefisso utilizzato in teoria dell'informazione per codificare simboli con una certa distribuzione di probabilità nota, in sequenze di bit dette parole di codice di lunghezza variabile. Ha la caratteristica di essere ottimo, ovvero di minimizzare la lunghezza media dei codici generati.

Essendo un codice a prefisso, il codice di Huffman garantisce che nessuna parola di codice associata ad un simbolo sia prefisso di un'altra parola associata ad un altro simbolo. Questo aspetto aiuta in fase di decodifica perché, appena si individua una parola di codice associata ad un simbolo, la si può decodificare immediatamente e senza ambiguità.

Le prestazioni di un codice di Huffman sono misurate in termini di lunghezza media dei codici generati ed è definita come la lunghezza media delle parole di codice, pesata in base alla loro probabilità:

$$L = \sum_{i=0}^{\# \text{ parole}} p_i \cdot l_i \quad \text{con} \quad \begin{array}{l} p_i = \text{probabilità di occorrenza del simbolo } i \\ l_i = \text{lunghezza della parola di codice associata al simbolo } i \end{array}$$

### Rappresentazione tramite Trie

Il trie è un albero binario pieno (ogni nodo interno ha due figli), non completo (le foglie possono essere a livelli diversi) utilizzato per rappresentare le parole di codice generate dal codice di Huffman. Ha le seguenti caratteristiche:

- ogni nodo interno del trie ha esattamente due figli, quello di sinistra raggiungibile tramite un arco etichettato con un bit 0 e quello di destra raggiungibile tramite un arco etichettato con un bit 1
- ogni foglia del trie rappresenta un simbolo, la cui parola di codice è data dalla concatenazione dei bit etichettati sugli archi del percorso che va dalla radice alla foglia

Si osservano le seguenti proprietà del trie:

- se una parola di codice fosse prefisso di un'altra, allora la parola più corta sarebbe rappresentata da un nodo interno del trie lungo il percorso che porta dalla radice alla foglia che rappresenta la parola più lunga; ciò è correttamente impossibile per come definito il trie
- si possono scambiare le due etichette 0, 1 degli archi che collegano i due figli al padre senza alterare le prestazioni del codice
- si possono scambiare simboli che si trovano allo stesso livello del trie senza alterare le prestazioni del codice, anche se hanno nodi padre diversi

### Algoritmo greedy e costruzione dell'albero

La costruzione dell'albero avviene in maniera bottom-up:

- si parte iniziando una foresta di  $n$  alberi, ognuno costituito da un singolo nodo foglia che rappresenta un simbolo con la relativa probabilità di occorrenza
- ad ogni passo si scelgono i due alberi con le probabilità della radice più piccole, e si crea un nuovo nodo padre comune alle due radici, con probabilità pari alla somma delle probabilità delle due radici; per convenzione si collega l'albero con probabilità più piccola alla sinistra assegnando un bit 0 all'arco, e l'albero con probabilità più grande a destra assegnando un bit 1 all'arco
- si ripete il passo precedente fino a quando tutte le foreste non si sono unite in un unico albero, che rappresenta il trie del codice di Huffman e la radice ha probabilità pari a 1

La scelta greedy è quella di prendere i due alberi le cui radici hanno le probabilità più piccole.

## Implementazione greedy iterativa con priority queue

0: **Algoritmo:** IT\_MCM( $\underline{c}, \underline{f}$ )

**Input:** vettori  $\underline{c}$  e  $\underline{f}$  rispettivamente degli  $n$  simboli e delle relative frequenze

**Output:** riferimento alla radice del trie che rappresenta il codice di Huffman

```
1: $n \leftarrow \text{len}(\underline{c})$
2: $Q \leftarrow \text{PriorityQueue}()$ ▷ creazione di una coda di priorità vuota
3: for all c in $\underline{c}$ do ▷ inizializzazione della coda di priorità con i nodi foglia
4: $z \leftarrow \text{new_node}(\text{symbol} : c, \text{freq} : \underline{f}[c], \text{left_node} : \text{null}, \text{right_node} : \text{null})$
5: $Q.\text{insert}(z.\text{freq}, z)$
6: for $i \leftarrow 1$ to $n - 1$ do ▷ ad ogni iterazione diminuisco di 1 il numero di alberi nella foresta
7: $x \leftarrow Q.\text{extract_min}()$ $y \leftarrow Q.\text{extract_min}()$
8: $z \leftarrow \text{new_node}(\text{symbol} : \text{null}, \text{freq} : x.\text{freq} + y.\text{freq}, \text{left_node} : x, \text{right_node} : y)$
9: $Q.\text{insert}(z.\text{freq}, z)$
10: return $Q.\text{extract_min}()$
```

## Analisi della complessità computazionale

Si sceglie di usare un heap binario per implementare la coda di priorità, che garantisce un costo  $\Theta(\log_2 n)$  per ogni inserimento ed estrazione. La complessità dell'algoritmo è data da:

- $n + n - 1 = 2n - 1$  inserimenti nella coda di priorità, con costo  $\Theta(\log_2 n)$  per ogni inserimento
- $2(n - 1) + 1 = 2n - 1$  estrazioni dalla coda di priorità, con costo  $\Theta(\log_2 n)$  per ogni estrazione
- costo lineare  $\Theta(n)$  dei due for, al netto delle operazioni di inserimento ed estrazione

$$T(n) = (2n - 1) \cdot T_{\text{insert}()} + (2n - 1) \cdot T_{\text{extract\_min}()} + \Theta(n) = \Theta(n \log_2 n)$$

Si osserva che, i primi  $n$  inserimenti delle foglie possono essere fatti in tempo  $\Theta(n)$  tramite l'algoritmo di costruzione di un heap dato un array di  $n$  elementi. Siccome, però, l'algoritmo di Huffman richiede almeno  $n$  estrazioni e non si dispone di algoritmi migliori di  $\Theta(\log_2 n)$  per l'estrazione, la complessità dell'algoritmo rimane  $\Theta(n \log_2 n)$ .

## Dimostrazione della correttezza

Di seguito si dimostra che la scelta greedy di estrarre i due nodi di frequenza minima  $a$  e  $b$  per primi per posizionali al livello più basso dell'albero, porta ad una soluzione ottima globale.

- si considera un albero ottimo  $T$  con costo ottimo  $B(T) = \sum_{i \in T} f(i) \cdot d(i) \leq B(T^*)$  minore di qualsiasi altro albero  $T^*$  in cui:
  - due foglie  $a$  e  $b$  che si trovano alla massima profondità dell'albero, uniti da un nodo padre  $p$
  - due foglie  $x$  e  $y$  con frequenze minime  $\max\{f(x), f(y)\} \leq \min\{f(a), f(b)\}$
- se  $a \equiv x$  e  $b \equiv y$ , la scelta greedy è banalmente rispettata
- altrimenti si ipotizza di scambiare  $a$  con  $x$  e  $b$  con  $y$ , ottenendo un nuovo albero  $T'$  e provocando una variazione del costo dell'albero  $\Delta B = B(T') - B(T)$  pari a:

$$\begin{aligned} \Delta B &= (f(a)d(x) + f(b)d(y) + f(x)d(a) + f(y)d(b)) - (f(a)d(a) + f(b)d(b) + f(x)d(x) + f(y)d(y)) \\ &= f(a)(d(x) - d(a)) + f(y)(d(b) - d(y)) + f(x)(d(a) - d(x)) + f(b)(d(y) - d(b)) \\ &= \underbrace{(f(a) - f(x))}_{\geq 0} \underbrace{(d(x) - d(a))}_{\leq 0} + \underbrace{(f(b) - f(y))}_{\geq 0} \underbrace{(d(y) - d(b))}_{\leq 0} \leq 0 \quad \Rightarrow \quad B(T') \leq B(T) \end{aligned}$$

- da  $B(T) \leq B(T')$  per l'ipotesi iniziale e  $B(T') \leq B(T)$  per il punto sopra, si ha che  $B(T) = B(T')$ , ovvero che anche l'albero  $T'$  che soddisfa la proprietà greedy è ottimo.

È utile notare che la scelta greedy non distingue se i nodi contenuti nella priority queue siano foglie o radici di sottoalberi, ma si basa solo sulla loro frequenza. Ciò significa che la dimostrazione sopra (che lavora solo con le foglie) vale anche per i nodi interni ad un certo livello dell'albero (che eventualmente potrebbero essere sostituiti da foglie di pari frequenza senza modificare il costo dell'albero).

## 10.4 (2,1)-boxing

### Formalizzazione del problema

Sia data una sequenza di oggetti  $O = \{o_1, o_2, \dots, o_n\}$  con la sequenza dei rispettivi pesi  $P = \{p_1, p_2, \dots, p_n\}$  positivi e minori di 1, ordinati in peso crescente ( $0 < p_1 \leq p_2 \leq \dots \leq p_n \leq 1$ ), e sia data inoltre una sequenza di scatole  $B = \{b_1, b_2, \dots, b_m\}$ , tali che ogni scatola  $b_p = \{o_i, o_j\}$  può contenere al massimo due oggetti  $o_i, o_j$  di peso massimo complessivo inferiore ad 1 ( $p_i + p_j \leq 1$ ).

Il problema del (2,1)-boxing consiste nel trovare una disposizione degli oggetti  $O$  nelle scatole  $B$  che minimizzi il numero di scatole utilizzate.

### Definizione della scelta greedy

Si sceglie di inserire per ogni scatola l'oggetto più pesante  $o_j$  non ancora assegnato (l'ultimo della sottostringa  $O_{i:j}$  degli oggetti non ancora assegnati) con eventuale aggiunta del più leggero  $o_i$  non ancora assegnato (il primo della sottostringa  $O_{i:j}$ ) se il peso lo permette.

Di seguito si formalizza la scelta greedy attraverso la proprietà di sottostruttura espressa in termini di equazioni alle ricorrenze.

$$B(O_{i:j}, P_{i:j}) = \begin{cases} \{b_p = \{o_i\}\} & \text{se } i = j \\ \{b_p = \{o_j\}\} \cup B(O_{i:j-1}, P_{i:j-1}) & \text{se } i < j \text{ e } p_i + p_j > 1 \\ \{b_p = \{o_i, o_j\}\} \cup B(O_{i+1:j-1}, P_{i+1:j-1}) & \text{se } i < j \text{ e } p_i + p_j \leq 1 \end{cases}$$

### Dimostrazione della correttezza

Per prima cosa si osserva che la scelta è fattibile, ovvero soddisfa il limite massimo di peso per ogni scatola imposto dal problema in tutti e tre i casi della definizione ricorsiva della scelta greedy.

Si dimostra ora che esiste una soluzione ottima che soddisfa la scelta greedy.

- si ipotizza una soluzione ottima  $B^*(O_{i:j}, P_{i:j})$  per l'istanza  $O_{i:j}, P_{i:j}$  e si analizza la scatola contenente l'oggetto più pesante  $b_p = \{o_j, \dots\}$ :
- se la scatola contiene solo l'oggetto più pesante  $b_p = \{o_j\}$  e vale  $p_i + p_j > 1$ , allora la scelta greedy è banalmente rispettata
- se la scatola contiene solo l'oggetto più pesante  $b_p = \{o_j\}$ , ma vale  $p_i + p_j \leq 1$ , allora è possibile spostare l'oggetto più leggero  $o_i$  dalla scatola in cui è inserito alla scatola  $b_p$  senza violare il vincolo di peso massimo e senza modificare il numero di scatole utilizzate, ottenendo una nuova soluzione ottima che rispetta la scelta greedy
- se la scatola contiene sia l'oggetto più pesante che quello più leggero  $b_p = \{o_i, o_j\}$ , allora la scelta greedy è banalmente rispettata siccome deve necessariamente valere  $p_i + p_j \leq 1$
- se la scatola contiene l'oggetto più pesante e un oggetto che non è il più leggero  $b_p = \{o_j, o_k\}$ , e l'oggetto più leggero si trova in un'altra scatola  $b_q = \{o_i, o_h\}$  allora è possibile scambiare  $o_i$  con  $o_k$  senza violare il vincolo di peso massimo (come illustrato di seguito) e senza modificare il numero di scatole utilizzate, ottenendo una nuova soluzione ottima that rispetta la scelta greedy

$$b_p = \{o_j, o_k\} \Rightarrow \begin{cases} p_j + p_k \leq 1 \Rightarrow p_j + p_i \leq p_j + p_k \leq 1 & \text{per } p_i \leq p_k \text{ con } o_i \text{ il più leggero} \\ p_j + p_k \leq 1 \Rightarrow p_h + p_k \leq p_j + p_k \leq 1 & \text{per } p_h \leq p_j \text{ con } o_j \text{ il più pesante} \end{cases}$$